Alien Invasion

v0.1.0

Generated by Doxygen 1.11.0

1 README 1

2 Hierarchical Index 3

2.1 Class Hierarchy . . . 3

3 Class Index 5

3.1 Class List . . . 5

4 File Index 7

4.1 File List . . . 7

5 Class Documentation 11

5.1 Bullet Class Reference . . . 11

5.1.1 Constructor & Destructor Documentation . . . 12

5.1.1.1 Bullet() . . . 12

5.1.2 Member Function Documentation . . . 12

5.1.2.1 Move() . . . 12

5.1.2.2 Render() . . . 12

5.1.2.3 setDamage() . . . 12

5.1.2.4 setTexture() . . . 13

5.1.2.5 Update() . . . 13

5.2 Button Class Reference . . . 13

5.2.1 Detailed Description . . . 14

5.2.2 Constructor & Destructor Documentation . . . 14

5.2.2.1 Button() . . . 14

5.2.3 Member Function Documentation . . . 15

5.2.3.1 changeColor() . . . 15

5.2.3.2 getText() . . . 16

5.2.3.3 isClicked() . . . 16

5.2.3.4 render() . . . 16

5.2.3.5 setOnClick() . . . 16

5.2.3.6 update() . . . 17

5.3 DarkMatter Class Reference . . . 17

5.3.1 Constructor & Destructor Documentation . . . 18

5.3.1.1 DarkMatter() . . . 18

5.3.2 Member Function Documentation . . . 19

5.3.2.1 Update() . . . 19

5.4 DynamicArray< T > Class Template Reference . . . 19

5.5 Enemy Class Reference . . . 20

5.5.1 Constructor & Destructor Documentation . . . 21

5.5.1.1 Enemy() . . . 21

5.5.2 Member Function Documentation . . . 21

5.5.2.1 Move() . . . 21

5.5.2.2 Render() . . . 21

5.5.2.3 TakeDamage() . . . 22
5.5.2.4 Update() . . . 23
5.6 FollowEnemy Class Reference . . . 23
5.6.1 Constructor & Destructor Documentation . . . 25
5.6.1.1 FollowEnemy() . . . 25
5.6.2 Member Function Documentation . . . 25
5.6.2.1 Move() . . . 25
5.7 Game Class Reference . . . 26
5.7.1 Constructor & Destructor Documentation . . . 28
5.7.1.1 Game() . . . 28
5.7.1.2 ∼Game() . . . 29
5.7.2 Member Function Documentation . . . 29
5.7.2.1 createPlayer() . . . 29
5.7.2.2 exit() . . . 30
5.7.2.3 getGameState() . . . 30
5.7.2.4 initUI() . . . 31
5.7.2.5 render() . . . 32
5.7.2.6 renderUI() . . . 33
5.7.2.7 restart() . . . 33
5.7.2.8 update() . . . 33
5.7.2.9 updateUI() . . . 34
5.8 LaserBullet Class Reference . . . 35
5.8.1 Constructor & Destructor Documentation . . . 36
5.8.1.1 LaserBullet() . . . 36
5.8.2 Member Function Documentation . . . 36
5.8.2.1 Update() . . . 36
5.9 Lightning Class Reference . . . 37
5.9.1 Constructor & Destructor Documentation . . . 38
5.9.1.1 Lightning() . . . 38
5.9.2 Member Function Documentation . . . 38
5.9.2.1 Update() . . . 38
5.10 Menu Class Reference . . . 39
5.10.1 Constructor & Destructor Documentation . . . 39
5.10.1.1 Menu() . . . 39
5.10.2 Member Function Documentation . . . 40
5.10.2.1 addButton() . . . 40
5.10.2.2 addButtonToBeginMenu() . . . 40
5.10.2.3 isButtonPressed() . . . 40
5.10.2.4 render() . . . 41
5.10.2.5 resetAllColor() . . . 41
5.10.2.6 resetColor() . . . 41
5.10.2.7 resetLowerLevelColor() . . . 42

5.10.2.8 resetUpperLevelColor() . . . 42
5.10.2.9 update() . . . 42
5.11 MoveLeftEnemy Class Reference . . . 42
5.11.1 Constructor & Destructor Documentation . . . 44
5.11.1.1 MoveLeftEnemy() . . . 44
5.11.2 Member Function Documentation . . . 44
5.11.2.1 Move() . . . 44
5.12 NuclierMaterial Class Reference . . . 45
5.12.1 Constructor & Destructor Documentation . . . 46
5.12.1.1 NuclierMaterial() . . . 46
5.12.2 Member Function Documentation . . . 46
5.12.2.1 Update() . . . 46
5.13 PeaBullet Class Reference . . . 46
5.13.1 Constructor & Destructor Documentation . . . 47
5.13.1.1 PeaBullet() . . . 47
5.13.2 Member Function Documentation . . . 48
5.13.2.1 Update() . . . 48
5.14 PeaShooter Class Reference . . . 48
5.14.1 Constructor & Destructor Documentation . . . 49
5.14.1.1 PeaShooter() . . . 49
5.14.1.2 ∼PeaShooter() . . . 49
5.14.2 Member Function Documentation . . . 50
5.14.2.1 Fire() . . . 50
5.14.2.2 Update() . . . 50
5.14.2.3 Upgrade() . . . 50
5.15 PlanetaryBomb Class Reference . . . 51
5.15.1 Constructor & Destructor Documentation . . . 52
5.15.1.1 PlanetaryBomb() . . . 52
5.15.2 Member Function Documentation . . . 52
5.15.2.1 Update() . . . 52
5.16 Plasma Class Reference . . . 52
5.16.1 Constructor & Destructor Documentation . . . 53
5.16.1.1 Plasma() . . . 53
5.16.2 Member Function Documentation . . . 54
5.16.2.1 Update() . . . 54
5.17 Player Class Reference . . . 54
5.17.1 Constructor & Destructor Documentation . . . 56
5.17.1.1 Player() . . . 56
5.17.2 Member Function Documentation . . . 57
5.17.2.1 addWeapon() . . . 57
5.17.2.2 CombatUpdate() . . . 57
5.17.2.3 getBullet() . . . 57

5.17.2.4 Move() . . . 57
5.17.2.5 removeBullet() . . . 58
5.17.2.6 Render() . . . 58
5.17.2.7 setBulletType() . . . 58
5.17.2.8 TakeDamage() . . . 59
5.17.2.9 Update() . . . 59
5.17.2.10 UpdateAccessories() . . . 59
5.17.2.11 updateLevelingSystem() . . . 60
5.18 TextTag Class Reference . . . 60
5.18.1 Constructor & Destructor Documentation . . . 60
5.18.1.1 TextTag() [1/2] . . . 60
5.18.1.2 TextTag() [2/2] . . . 61
5.18.2 Member Function Documentation . . . 61
5.18.2.1 operator=() . . . 61
5.18.2.2 Render() . . . 61
5.18.2.3 Update() . . . 62
5.19 Weapon Class Reference . . . 62
5.19.1 Constructor & Destructor Documentation . . . 63
5.19.1.1 Weapon() . . . 63
5.19.2 Member Function Documentation . . . 63
5.19.2.1 Fire() . . . 63
5.19.2.2 Render() . . . 64
5.19.2.3 Update() . . . 64
5.19.2.4 Upgrade() . . . 64

6 File Documentation **65**
6.1 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Bullet.hpp File Reference . . . 65
6.1.1 Detailed Description . . . 65
6.2 Bullet.hpp . . . 66
6.3 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/DarkMatter.hpp File Reference . . . 66
6.3.1 Detailed Description . . . 66
6.4 DarkMatter.hpp . . . 67
6.5 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/LaserBullet.hpp File Reference . . . 67
6.5.1 Detailed Description . . . 68
6.6 LaserBullet.hpp . . . 68
6.7 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Lightning.hpp File Reference . . . 68
6.7.1 Detailed Description . . . 69
6.8 Lightning.hpp . . . 69
6.9 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/NuclierMaterial.hpp File Reference . . . 69

6.9.1 Detailed Description . . . 70
6.10 NuclierMaterial.hpp . . . 70
6.11 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PeaBullet.hpp File Reference . . . 70
6.11.1 Detailed Description . . . 71
6.12 PeaBullet.hpp . . . 71
6.13 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PlanetaryBomb.hpp File Reference . . . 71
6.13.1 Detailed Description . . . 72
6.14 PlanetaryBomb.hpp . . . 72
6.15 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Plasma.hpp File Reference . . . 72
6.15.1 Detailed Description . . . 73
6.16 Plasma.hpp . . . 73
6.17 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Enemy.hpp File Reference . . . 73
6.17.1 Detailed Description . . . 74
6.18 Enemy.hpp . . . 74
6.19 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Follow.hpp File Reference . . . 75
6.19.1 Detailed Description . . . 75
6.20 Follow.hpp . . . 76
6.21 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/MoveLeft.hpp File Reference . . . 76
6.21.1 Detailed Description . . . 76
6.22 MoveLeft.hpp . . . 77
6.23 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Game.hpp File Reference . . . 77
6.23.1 Detailed Description . . . 77
6.24 Game.hpp . . . 78
6.25 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Menu.hpp File Reference . . . 80
6.25.1 Detailed Description . . . 80
6.26 Menu.hpp . . . 81
6.27 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Player.hpp File Reference . . . 81
6.27.1 Detailed Description . . . 82
6.28 Player.hpp . . . 83
6.29 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/BulletResources.hpp File Reference . . . 85
6.29.1 Detailed Description . . . 86
6.29.2 Variable Documentation . . . 86
6.29.2.1 resourcePathsBulletsDamage . . . 86
6.29.2.2 resourcePathsDamageMaps . . . 86
6.29.2.3 resourcePathsDarkMatterBullet . . . 87

6.29.2.4 resourcePathsDarkMatterBulletDamage . . . 87
6.29.2.5 resourcePathsLaserBullets . . . 87
6.29.2.6 resourcePathsLightningBullets . . . 87
6.29.2.7 resourcePathsLightningBulletsDamage . . . 87
6.29.2.8 resourcePathsMaps . . . 88
6.29.2.9 resourcePathsNuclierMaterialBullet . . . 88
6.29.2.10 resourcePathsNuclierMaterialBulletDamage . . . 88
6.29.2.11 resourcePathsPeaShooterBullets . . . 88
6.29.2.12 resourcePathsPeaShooterBulletsDamage . . . 88
6.29.2.13 resourcePathsPlanetaryBombBullets . . . 89
6.29.2.14 resourcePathsPlanetaryBombBulletsDamage . . . 89
6.29.2.15 resourcePathsPlasmaBullets . . . 89
6.29.2.16 resourcePathsPlasmaBulletsDamage . . . 89
6.30 BulletResources.hpp . . . 90
6.31 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/↩ Button.hpp File Reference . . . 92
6.31.1 Detailed Description . . . 92
6.32 Button.hpp . . . 93
6.33 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/↩ DynamicArray.hpp File Reference . . . 93
6.33.1 Detailed Description . . . 94
6.34 DynamicArray.hpp . . . 94
6.35 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/↩ Fonts.hpp File Reference . . . 96
6.35.1 Detailed Description . . . 96
6.35.2 Variable Documentation . . . 97
6.35.2.1 resourcePathsFonts . . . 97
6.36 Fonts.hpp . . . 97
6.37 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/↩ SaveData.hpp File Reference . . . 97
6.37.1 Detailed Description . . . 99
6.37.2 Function Documentation . . . 99
6.37.2.1 eraceSavedData() . . . 99
6.37.2.2 loadSavedData() . . . 100
6.37.2.3 saveData() . . . 101
6.37.2.4 saveDefaultData() . . . 101
6.37.2.5 setContinueGame() . . . 102
6.37.2.6 setLowerWeapon() . . . 102
6.37.2.7 setLowerWeaponLevel() . . . 102
6.37.2.8 setNumberOfPlayers() . . . 102
6.37.2.9 setPlayer1Exp() . . . 103
6.37.2.10 setPlayer1ExpNext() . . . 103
6.37.2.11 setPlayer1Hp() . . . 103

6.37.2.12 setPlayer1HpMax() . . . 103
6.37.2.13 setPlayer1Level() . . . 104
6.37.2.14 setPlayer1Score() . . . 104
6.37.2.15 setPlayer2Exp() . . . 104
6.37.2.16 setPlayer2ExpNext() . . . 104
6.37.2.17 setPlayer2Hp() . . . 105
6.37.2.18 setPlayer2HpMax() . . . 105
6.37.2.19 setPlayer2Level() . . . 105
6.37.2.20 setPlayer2Score() . . . 105
6.37.2.21 setTypeOfBullet() . . . 106
6.37.2.22 setUpperWeapon() . . . 106
6.37.2.23 setUpperWeaponLevel() . . . 106
6.38 SaveData.hpp . . . 107
6.39 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/↩ Textures.hpp File Reference . . . 107
6.39.1 Detailed Description . . . 108
6.39.2 Variable Documentation . . . 108
6.39.2.1 PeaShooterDown . . . 108
6.39.2.2 PeaShooterUp . . . 108
6.39.2.3 resourcePaths . . . 109
6.39.2.4 SideGunsDown . . . 109
6.39.2.5 SideGunsUp . . . 109
6.40 Textures.hpp . . . 109
6.41 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/TextTag.hpp File Reference . . . 110
6.41.1 Detailed Description . . . 111
6.42 TextTag.hpp . . . 111
6.43 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/↩ PeaShooter.hpp File Reference . . . 112
6.43.1 Detailed Description . . . 112
6.44 PeaShooter.hpp . . . 112
6.45 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/↩ Weapon.hpp File Reference . . . 113
6.45.1 Detailed Description . . . 113
6.46 Weapon.hpp . . . 113
6.47 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Bullet.cpp File Reference . . . 114
6.47.1 Detailed Description . . . 114
6.48 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Dark↩ Matter.cpp File Reference . . . 115
6.48.1 Detailed Description . . . 115
6.49 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Laser↩ Bullet.cpp File Reference . . . 115
6.49.1 Detailed Description . . . 115

6.50 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Lightning.cpp File Reference . . . 116
6.50.1 Detailed Description . . . 116
6.51 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Nuclier↩ Material.cpp File Reference . . . 116
6.51.1 Detailed Description . . . 116
6.52 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Pea↩ Bullet.cpp File Reference . . . 117
6.52.1 Detailed Description . . . 117
6.53 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Planetary↩ Bomb.cpp File Reference . . . 117
6.53.1 Detailed Description . . . 117
6.54 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Plasma.cpp File Reference . . . 118
6.54.1 Detailed Description . . . 118
6.55 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Enemy.cpp File Reference . . . 118
6.55.1 Detailed Description . . . 118
6.56 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Follow.cpp File Reference . . . 119
6.56.1 Detailed Description . . . 119
6.57 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Move↩ Left.cpp File Reference . . . 119
6.57.1 Detailed Description . . . 120
6.58 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Game.cpp File Reference . . . 120
6.58.1 Detailed Description . . . 120
6.59 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/main.cpp File Reference . . . 121
6.59.1 Detailed Description . . . 121
6.59.2 Function Documentation . . . 122
6.59.2.1 main() . . . 122
6.60 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Menu.cpp File Reference . . . 122
6.60.1 Detailed Description . . . 123
6.61 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Player.cpp File Reference . . . 123
6.61.1 Detailed Description . . . 123
6.62 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Resources/↩ Button.cpp File Reference . . . 124
6.62.1 Detailed Description . . . 124
6.63 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Resources/Save↩ Data.cpp File Reference . . . 124
6.63.1 Detailed Description . . . 126
6.63.2 Function Documentation . . . 126
6.63.2.1 eraceSavedData() . . . 126

6.63.2.2 loadSavedData() . . . 127
6.63.2.3 readJsonFromFile() . . . 128
6.63.2.4 saveData() . . . 129
6.63.2.5 saveDefaultData() . . . 130
6.63.2.6 setContinueGame() . . . 130
6.63.2.7 setLowerWeapon() . . . 130
6.63.2.8 setLowerWeaponLevel() . . . 130
6.63.2.9 setNumberOfPlayers() . . . 130
6.63.2.10 setPlayer1Exp() . . . 131
6.63.2.11 setPlayer1ExpNext() . . . 131
6.63.2.12 setPlayer1Hp() . . . 131
6.63.2.13 setPlayer1HpMax() . . . 131
6.63.2.14 setPlayer1Level() . . . 132
6.63.2.15 setPlayer1Score() . . . 132
6.63.2.16 setPlayer2Exp() . . . 132
6.63.2.17 setPlayer2ExpNext() . . . 132
6.63.2.18 setPlayer2Hp() . . . 133
6.63.2.19 setPlayer2HpMax() . . . 133
6.63.2.20 setPlayer2Level() . . . 133
6.63.2.21 setPlayer2Score() . . . 133
6.63.2.22 setTypeOfBullet() . . . 134
6.63.2.23 setUpperWeapon() . . . 134
6.63.2.24 setUpperWeaponLevel() . . . 134
6.63.2.25 writeJsonToFile() . . . 134
6.64 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/TextTag.cpp File Reference . . . 135
6.64.1 Detailed Description . . . 135
6.65 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/PeaShooter.cpp File Reference . . . 135
6.65.1 Detailed Description . . . 136
6.66 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/Weapon.cpp File Reference . . . 136
6.66.1 Detailed Description . . . 136

Index **137**

Chapter 1

README

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Bullet . . . 11
  - DarkMatter . . . 17
  - LaserBullet . . . 35
  - Lightning . . . 37
  - NuclierMaterial . . . 45
  - PeaBullet . . . 46
  - PlanetaryBomb . . . 51
  - Plasma . . . 52
- Button . . . 13
- DynamicArray< T > . . . 19
- DynamicArray< Bullet * > . . . 19
- DynamicArray< Enemy * > . . . 19
- DynamicArray< TextTag > . . . 19
- DynamicArray< Texture > . . . 19
- DynamicArray< Weapon * > . . . 19
- Enemy . . . 20
  - FollowEnemy . . . 23
  - MoveLeftEnemy . . . 42
- Game . . . 26
- Menu . . . 39
- Player . . . 54
- TextTag . . . 60
- Weapon . . . 62
  - PeaShooter . . . 48

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Bullet 11
Button
A class to represent a button in the game 13
DarkMatter 17
DynamicArray< T > 19
Enemy 20
FollowEnemy 23
Game 26
LaserBullet 35
Lightning 37
Menu 39
MoveLeftEnemy 42
NuclierMaterial 45
PeaBullet 46
PeaShooter 48
PlanetaryBomb 51
Plasma 52
Player 54
TextTag 60
Weapon 62

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Game.hpp
This is the header file for the Game class, which is used to create the game and manage the game state . . . 77
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Menu.hpp
This is the header file for the Menu class, which is used to create the main menu of the game . 80
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Player.hpp
This is the header file for the Player class, which is used to create the player object . . . 81
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/TextTag.hpp
This is the header file for the TextTag class, which is used to create text tags that can be displayed on the screen . . . 110
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Bullet.hpp
This is the header file for the Bullet class, which is used to create bullets for the player to shoot 65
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/DarkMatter.hpp
This is the header file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot . . . 66
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/LaserBullet.hpp
This is the header file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot . . . 67
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Lightning.hpp
This is the header file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot . . . 68
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/NuclierMaterial.hpp
This is the header file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot . . . 69
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PeaBullet.hpp
This is the header file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot . . . 70
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PlanetaryBomb.hpp
This is the header file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot . . . 71
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Plasma.hpp
This is the header file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot . . . 72
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Enemy.hpp
This is the header file for the Enemy class, which is used to create enemies for the player to fight against . . . 73

/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Follow.hpp
This is the header file for the FollowEnemy class, child of Enemy, which is used to create enemies that follow the player . . . 75
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/MoveLeft.hpp
This is the header file for the MoveLeftEnemy class, child of Enemy, which is used to create enemies that move left . . . 76
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/BulletResources.hpp
This is the header file for the BulletResources enum, which is used to store the paths to the bullet sprites and their damage values . . . 85
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Button.hpp
This is the header file for the Button class, which is used to create buttons for the game . . . 92
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/DynamicArray.hpp
This is the header file for the DynamicArray class, which is used to create a dynamic array of elements of any type . . . 93
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Fonts.hpp
This is the header file for the Fonts enum, which is used to store the paths to the fonts used in the game . . . 96
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/SaveData.hpp
This is the header file for the SaveData class, which is used to save and load data from the game 97
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Textures.hpp
This is the header file for the Textures enum, which is used to store the paths to the textures used in the game . . . 107
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/PeaShooter.hpp
This is the header file for the PeaShooter class, child of Weapon, which is used to create PeaShooter weapons for the player to shoot . . . 112
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/Weapon.hpp
This is the header file for the Weapon class, which is used to create weapons for the player to shoot . . . 113
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Game.cpp
This is the implementation file for the Game class, which is used to manage the game logic . . 120
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/main.cpp
This is the main file for the game . . . 121
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Menu.cpp
This is the implementation file for the Menu class, which is used to create and manage the game menu . . . 122
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Player.cpp
Implementation file for the Player class . . . 123
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/TextTag.cpp
Implementation file for the TextTag class . . . 135
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Bullet.cpp
This is the implementation file for the Bullet class, which is used to create bullets for the player to shoot . . . 114
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/DarkMatter.cpp
This is the implementation file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot . . . 115
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/LaserBullet.cpp
This is the implementation file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot . . . 115
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Lightning.cpp
This is the implementation file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot . . . 116
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/NuclierMaterial.cpp
This is the implementation file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot . . . 116
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/PeaBullet.cpp
This is the implementation file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot . . . 117

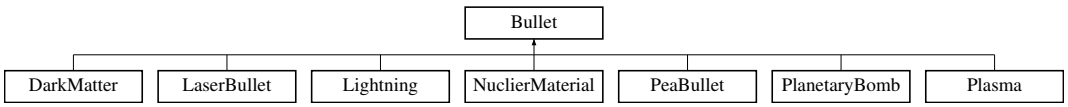
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/PlanetaryBomb.cpp
This is the implementation file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot 117
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Plasma.cpp
This is the implementation file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot 118
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Enemy.cpp
This is the implementation file for the Enemy class, which is used to create enemies for the player to fight against 118
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Follow.cpp
This is the implementation file for the FollowEnemy class, a subclass of Enemy, used to create enemies that follow the player 119
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/MoveLeft.cpp
This is the implementation file for the MoveLeftEnemy class, a subclass of Enemy, used to create enemies that move left 119
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Resources/Button.cpp
This is the implementation file for the Button class, which is used to create buttons for the game 124
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Resources/SaveData.cpp
This is the implementation file for the SaveData functions, which are used to save and load game data 124
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/PeaShooter.cpp
This is the implementation file for the PeaShooter class, a child of Weapon, used to create the pea shooter weapon for the player 135
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/Weapon.cpp
This is the implementation file for the Weapon class, which is used to create weapons for the player 136

Chapter 5

Class Documentation

5.1 Bullet Class Reference

Inheritance diagram for Bullet:



Public Member Functions

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- virtual void Update (const float &dt)

  Updates the bullet's movement.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Protected Attributes

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.1.1 Constructor & Destructor Documentation

5.1.1.1 Bullet()

```
Bullet::Bullet (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new Bullet object.

Parameters

pos	Initial position of the bullet.
level	Level of the bullet.
direction	Direction of the bullet.
initialVelocity	Initial velocity of the bullet.
maxVelocity	Maximum velocity of the bullet.
acceleration	Acceleration of the bullet.

5.1.2 Member Function Documentation

5.1.2.1 Move()

```
void Bullet::Move (
            const float & dt)  [virtual]
```

Moves the bullet based on its current velocity.

Parameters

dt	Time elapsed since the last update.

5.1.2.2 Render()

```
void Bullet::Render (
            RenderTarget & target)
```

Renders the bullet on the target.

Parameters

target	The render target.

5.1.2.3 setDamage()

```
void Bullet::setDamage (
            int type,
            int level)
```

Sets the damage of the bullet based on its type and level.

Parameters

type	Type of the bullet.
level	Level of the bullet.

5.1.2.4 setTexture()

```
void Bullet::setTexture (
            int type,
            int level)
```

Sets the texture of the bullet based on its type and level.

Parameters

type	Type of the bullet.
level	Level of the bullet.

5.1.2.5 Update()

```
void Bullet::Update (
            const float & dt)  [virtual]
```

Updates the bullet's movement.

Parameters

dt	Time elapsed since the last update.

Reimplemented in DarkMatter, LaserBullet, Lightning, NuclierMaterial, PeaBullet, PlanetaryBomb, and Plasma.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Bullet.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Bullet.cpp

5.2 Button Class Reference

A class to represent a button in the game.

```
#include <Button.hpp>
```

Public Member Functions

- Button (float x, float y, float width, float height, float radius, sf::Font &font, const std::string &text, const sf::Color &idleColor, const sf::Color &hoverColor, const sf::Color &activeColor)

  Construct a new Button object.
- void setOnClick (std::function< void()> func)

  Set the callback function to be called when the button is clicked.
- bool isClicked (const sf::Vector2f &mousePos) const

  Check if the button is clicked based on the mouse position.
- const std::string getText () const

  Get the text displayed on the button.
- void update (const sf::Vector2f &mousePos)

  Update the button's state based on the mouse position.
- void render (sf::RenderTarget &target)

  Render the button on the target.
- void changeColor (const sf::Color &idleColor, const sf::Color &hoverColor, const sf::Color &activeColor)

  Change the button's colors for different states.

Public Attributes

- int **timesPressed** = 0

5.2.1 Detailed Description

A class to represent a button in the game.

This class encapsulates the properties and behaviors of a button, including rendering, updating its state based on user interactions, and handling click events.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Button()

```
Button::Button (
            float x,
            float y,
            float width,
            float height,
            float radius,
            sf::Font & font,
            const std::string & text,
            const sf::Color & idleColor,
            const sf::Color & hoverColor,
            const sf::Color & activeColor)
```

Construct a new Button object.

Constructs a Button object.

Parameters

x	X coordinate of the button's position.
y	Y coordinate of the button's position.
width	Width of the button.
height	Height of the button.
radius	Radius for the button's corners.
font	Font used for the button's text.
text	Text displayed on the button.
idleColor	Color of the button when idle.
hoverColor	Color of the button when hovered.
activeColor	Color of the button when active (pressed).

Initializes the Button object with specified parameters.

Parameters

x	The x-coordinate of the button position.
y	The y-coordinate of the button position.
width	The width of the button.
height	The height of the button.
radius	The corner radius of the button.
font	Reference to the font used for the button text.
text	The text displayed on the button.
idleColor	The color of the button in idle state.
hoverColor	The color of the button when hovered over.
activeColor	The color of the button when clicked.

Adjust the vertical offset by changing the Y position of the text

5.2.3 Member Function Documentation

5.2.3.1 changeColor()

```
void Button::changeColor (
            const sf::Color & idleColor,
            const sf::Color & hoverColor,
            const sf::Color & activeColor)  [inline]
```

Change the button's colors for different states.

Parameters

idleColor	New idle color.
hoverColor	New hover color.
activeColor	New active color.

5.2.3.2 getText()

```
const std::string Button::getText () const  [inline]
```

Get the text displayed on the button.

Returns

The text displayed on the button as a std::string.

5.2.3.3 isClicked()

```
bool Button::isClicked (
            const sf::Vector2f & mousePos) const  [inline]
```

Check if the button is clicked based on the mouse position.

Parameters

mousePos	Position of the mouse cursor.

Returns

true If the button is clicked.

false If the button is not clicked.

5.2.3.4 render()

```
void Button::render (
            sf::RenderTarget & target)
```

Render the button on the target.

Renders the button.

Parameters

target	The render target to draw the button on.

Draws the button shape and text to the specified render target.

Parameters

target	The render target to draw the button on.

5.2.3.5 setOnClick()

```
void Button::setOnClick (
            std::function< void() > func)
```

Set the callback function to be called when the button is clicked.

Sets the function to execute when the button is clicked.

Parameters

func	A std::function object representing the callback function.
func	The function to execute when the button is clicked.

5.2.3.6 update()

```
void Button::update (
            const sf::Vector2f & mousePos)
```

Update the button's state based on the mouse position.

Updates the state of the button.

Parameters

mousePos	Position of the mouse cursor.

Checks if the button is hovered over or clicked, and updates its state accordingly.

Parameters

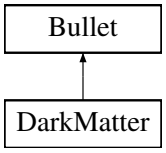
mousePos	The position of the mouse cursor.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Button.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Resources/Button.cpp

5.3 DarkMatter Class Reference

Inheritance diagram for DarkMatter:



Public Member Functions

- DarkMatter (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩Velocity=50.f, float acceleration=1.f)

  Constructs a new DarkMatter object.
- void Update (const float &dt) override

  Updates the DarkMatter bullet's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.3.1 Constructor & Destructor Documentation

5.3.1.1 DarkMatter()

```
DarkMatter::DarkMatter (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new DarkMatter object.

Parameters

pos	Initial position of the DarkMatter bullet.
level	Level of the DarkMatter bullet.
direction	Direction of the DarkMatter bullet.
initialVelocity	Initial velocity of the DarkMatter bullet.
maxVelocity	Maximum velocity of the DarkMatter bullet.
acceleration	Acceleration of the DarkMatter bullet.

5.3.2 Member Function Documentation

5.3.2.1 Update()

```
void DarkMatter::Update (
            const float & dt)  [override], [virtual]
```

Updates the DarkMatter bullet's movement.

Parameters

dt	Time elapsed since the last update.

Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/DarkMatter.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/DarkMatter.cpp

5.4 DynamicArray< T > Class Template Reference

Public Types

- typedef T ** **iterator**
- typedef const T ** **const_iterator**

Public Member Functions

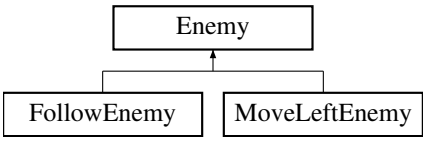
- **DynamicArray** (unsigned initialCapacity=5)
- **DynamicArray** (const DynamicArray &other)
- T & **operator[]** (int index)
- void **operator=** (const DynamicArray &other)
- void **add** (const T &element)
- void **remove** (int index)
- unsigned **size** () const
- void **clear** ()
- iterator **begin** ()
- iterator **end** ()
- const_iterator **begin** () const
- const_iterator **end** () const

The documentation for this class was generated from the following file:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/DynamicArray.hpp

5.5 Enemy Class Reference

Inheritance diagram for Enemy:



Public Member Functions

- Enemy (sf::Texture *texture, sf::Vector2u windowBounds, sf::Vector2f position, sf::Vector2f direction, sf::↩ Vector2f scale, float exp, int hpMax, int damageMax, int damageMin)

  Constructs a new Enemy object.
- virtual ~**Enemy** ()

  Destroys the Enemy object.
- const int **getDamage** () const
- const int **getHp** () const
- const int **getHpMax** () const
- const float **getExp** () const
- const bool **isDead** () const
- const sf::FloatRect **getBounds** () const
- const sf::Vector2f & **getPosition** () const
- const int **getPlayerToFollow** () const
- void TakeDamage (int damage)

  Inflicts damage on the enemy.
- virtual void Move (float dt, Vector2f playerPosition)=0
- virtual void Update (float dt, Vector2f playerPosition)

  Updates the state of the enemy.
- void Render (sf::RenderTarget &target)

  Renders the enemy on the screen.

Protected Attributes

- float **dtMultiplier**
- sf::Sprite **sprite**
- sf::Texture * **texture**
- sf::Vector2f **direction**
- sf::Vector2u **windowBounds**
- float **damageTimer**
- float **damageTimerMax**
- int **hp**
- int **hpMax**
- int **damageMin**
- int **damageMax**
- float **exp**
- int **enemySpawnTimer**
- int **enemySpawnTimerMax**
- int **playerToFollow** = 0
- float **speedx**
- float **speedy**

5.5.1 Constructor & Destructor Documentation

5.5.1.1 Enemy()

```
Enemy::Enemy (
            sf::Texture * texture,
            sf::Vector2u windowBounds,
            sf::Vector2f position,
            sf::Vector2f direction,
            sf::Vector2f scale,
            float exp,
            int hpMax,
            int damageMax,
            int damageMin)
```

Constructs a new Enemy object.

Parameters

texture	Pointer to the texture for the enemy.
windowBounds	The bounds of the game window.
position	Initial position of the enemy.
direction	Initial movement direction of the enemy.
scale	Scale factor for the enemy sprite.
exp	Experience points gained when defeating the enemy.
hpMax	Maximum hit points of the enemy.
damageMax	Maximum damage inflicted by the enemy.
damageMin	Minimum damage inflicted by the enemy.

5.5.2 Member Function Documentation

5.5.2.1 Move()

```
virtual void Enemy::Move (
            float dt,
            Vector2f playerPosition)  [pure virtual]
```

Implemented in FollowEnemy, and MoveLeftEnemy.

5.5.2.2 Render()

```
void Enemy::Render (
            sf::RenderTarget & target)
```

Renders the enemy on the screen.

Parameters

target	The render target.

5.5.2.3 TakeDamage()

```
void Enemy::TakeDamage (
            int damage)
```

Inflicts damage on the enemy.

Parameters

damage	Amount of damage to be inflicted.

5.5.2.4 Update()

```
void Enemy::Update (
            float dt,
            Vector2f playerPosition)  [virtual]
```

Updates the state of the enemy.

Parameters

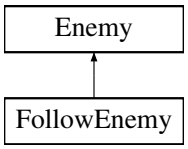
dt	Time elapsed since the last update.
playerPosition	Position of the player.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Enemy.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Enemy.cpp

5.6 FollowEnemy Class Reference

Inheritance diagram for FollowEnemy:



Public Member Functions

- FollowEnemy (Texture *texture, Vector2u windowBounds, Vector2f position, Vector2f direction, Vector2f scale, float exp, int hpMax, int damageMax, int damageMin, int playerToFollow=0)

  Constructs a new FollowEnemy object.
- virtual ~**FollowEnemy** ()

  Destroys the FollowEnemy object.
- void Move (float dt, Vector2f playerPosition) override

  Moves the FollowEnemy.

Public Member Functions inherited from Enemy

- Enemy (sf::Texture *texture, sf::Vector2u windowBounds, sf::Vector2f position, sf::Vector2f direction, sf::↩ Vector2f scale, float exp, int hpMax, int damageMax, int damageMin)

  Constructs a new Enemy object.
- virtual ~**Enemy** ()

  Destroys the Enemy object.
- const int **getDamage** () const
- const int **getHp** () const
- const int **getHpMax** () const
- const float **getExp** () const
- const bool **isDead** () const
- const sf::FloatRect **getBounds** () const
- const sf::Vector2f & **getPosition** () const
- const int **getPlayerToFollow** () const
- void TakeDamage (int damage)

  Inflicts damage on the enemy.
- virtual void Update (float dt, Vector2f playerPosition)

  Updates the state of the enemy.
- void Render (sf::RenderTarget &target)

  Renders the enemy on the screen.

Additional Inherited Members

Protected Attributes inherited from Enemy

- float **dtMultiplier**
- sf::Sprite **sprite**
- sf::Texture * **texture**
- sf::Vector2f **direction**
- sf::Vector2u **windowBounds**
- float **damageTimer**
- float **damageTimerMax**
- int **hp**
- int **hpMax**
- int **damageMin**
- int **damageMax**
- float **exp**
- int **enemySpawnTimer**
- int **enemySpawnTimerMax**
- int **playerToFollow** = 0
- float **speedx**
- float **speedy**

5.6.1 Constructor & Destructor Documentation

5.6.1.1 FollowEnemy()

```
FollowEnemy::FollowEnemy (
            Texture * texture,
            Vector2u windowBounds,
            Vector2f position,
            Vector2f direction,
            Vector2f scale,
            float exp,
            int hpMax,
            int damageMax,
            int damageMin,
            int playerToFollow = 0)
```

Constructs a new FollowEnemy object.

Parameters

texture	Pointer to the texture for the enemy.
windowBounds	The bounds of the game window.
position	Initial position of the enemy.
direction	Initial movement direction of the enemy.
scale	Scale factor for the enemy sprite.
exp	Experience points gained when defeating the enemy.
hpMax	Maximum hit points of the enemy.
damageMax	Maximum damage inflicted by the enemy.
damageMin	Minimum damage inflicted by the enemy.
playerToFollow	Index of the player to follow.

Duration to follow the player

5.6.2 Member Function Documentation

5.6.2.1 Move()

```
void FollowEnemy::Move (
            float dt,
            Vector2f playerPosition)  [override], [virtual]
```

Moves the FollowEnemy.

Parameters

dt	Time elapsed since the last update.
playerPosition	Position of the player.

Calculate direction towards the player

Normalize the direction

Calculate the angle to rotate the sprite

Move towards the player

Adjust speed as needed

Adjust speed as needed

Check if the enemy has passed the player

Reset rotation to face straight

Continue moving straight in the initial direction

Implements Enemy.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Follow.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/Follow.cpp

5.7 Game Class Reference

Public Member Functions

- Game (RenderWindow *window, int numberOfPlayers, int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score, int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score, bool upperWeapon, int upperWeaponLevel, bool lowerWeapon, int lowerWeaponLevel, int typeOfBullet)

  Constructor for the Game class.
- virtual ~Game ()

  Destructor for the Game class.
- RenderWindow & **getWindow** ()
- const int & **getNumberOfPlayers** () const
- const int & **getPlayer1Level** () const
- const int & **getPlayer2Level** () const
- const int & **getPlayer1Hp** () const
- const int & **getPlayer2Hp** () const
- const int & **getPlayer1HpMax** () const
- const int & **getPlayer2HpMax** () const
- const float & **getPlayer1Exp** () const
- const float & **getPlayer2Exp** () const
- const float & **getPlayer1ExpNext** () const
- const float & **getPlayer2ExpNext** () const
- const int & **getPlayer1Score** () const
- const int & **getPlayer2Score** () const

- const bool & **getUpperWeapon** () const
- const bool & **getLowerWeapon** () const
- const int & **getUpperWeaponLevel** () const
- const int & **getLowerWeaponLevel** () const
- const int **getBulletType** () const
- const bool & **isGameOver** () const
- const bool & **isGamePaused** () const
- const bool & **isInGame** () const
- bool **hasBeenReset** ()
- bool **isCountinue** ()
- void **pause** ()
- void **resume** ()
- void **setIsGameStarted** (bool isGameStarted)
- void **setWeaponType** (int weaponType)
- void **setUpperWeapon** (bool upperWeapon)
- void **setLowerWeapon** (bool lowerWeapon)
- void **setUpperWeaponLevel** (int upperWeaponLevel)
- void **setLowerWeaponLevel** (int lowerWeaponLevel)
- Player ∗ createPlayer (DynamicArray< Texture > &textures, Vector2u windowBounds, int level, float player↩Exp, float playerExpNext, int playerHp, int playerHpMax, int playerScore, int upKey, int downKey, int leftKey, int rightKey, int fireKey, int typeOfBullet)

  Create player.
- void getGameState (int &numberOfPlayers, int &player1Level, int &player1Hp, int &player1HpMax, float &player1Exp, float &player1ExpNext, int &player1Score, int &player2Level, int &player2Hp, int &player2↩HpMax, float &player2Exp, float &player2ExpNext, int &player2Score, bool &upperWeapon, int &upper↩WeaponLevel, bool &lowerWeapon, int &lowerWeaponLevel)

  Get the current game state including player information and weapon states.
- void **loadSavedGameData** (int numberOfPlayers, int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score, int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score, bool upperWeapon, int upperWeaponLevel, bool lower↩Weapon, int lowerWeaponLevel, int typeOfBullet)
- void **reseted** (bool reset)
- void restart ()

  Restart the game.
- void exit ()

  Exit the game.
- void initUI ()

  Initialize the UI elements.
- void updateUI (int index)

  Update the UI elements for a specified player.
- void update (const float &dt)

  Update the game state.
- void renderUI ()

  Render UI elements.
- void render ()

  Renders the game elements.

Public Attributes

- int **times** = 0
- int **timesPlayers** = 0

5.7.1 Constructor & Destructor Documentation

5.7.1.1 Game()

```
Game::Game (
            RenderWindow * window,
            int numberOfPlayers,
            int player1Level,
            int player1Hp,
            int player1HpMax,
            float player1Exp,
            float player1ExpNext,
            int player1Score,
            int player2Level,
            int player2Hp,
            int player2HpMax,
            float player2Exp,
            float player2ExpNext,
            int player2Score,
            bool upperWeapon,
            int upperWeaponLevel,
            bool lowerWeapon,
            int lowerWeaponLevel,
            int typeOfBullet)
```

Constructor for the Game class.

Initializes the game with the specified settings for the window, players, weapons, and game parameters.

Parameters

window	Pointer to the SFML RenderWindow.
numberOfPlayers	Number of players (1 or 2).
player1Level	Level of player 1.
player1Hp	Current health points of player 1.
player1HpMax	Maximum health points of player 1.
player1Exp	Current experience points of player 1.
player1ExpNext	Experience points needed for player 1 to level up.
player1Score	Current score of player 1.
player2Level	Level of player 2.
player2Hp	Current health points of player 2.
player2HpMax	Maximum health points of player 2.
player2Exp	Current experience points of player 2.
player2ExpNext	Experience points needed for player 2 to level up.
player2Score	Current score of player 2.
upperWeapon	Indicates if the upper weapon is enabled.
upperWeaponLevel	Level of the upper weapon.
lowerWeapon	Indicates if the lower weapon is enabled.
lowerWeaponLevel	Level of the lower weapon.
typeOfBullet	Type of bullet used by the player(s).

Window settings

Load resources

Create players

Initialize enemy spawn timer

Initialize UI

5.7.1.2 ~Game()

```
Game::~Game () [virtual]
```

Destructor for the Game class.

Cleans up dynamically allocated memory for players and enemies. Delete player objects

Delete enemy objects

Clear enemy and player vectors

5.7.2 Member Function Documentation

5.7.2.1 createPlayer()

```
Player * Game::createPlayer (
            DynamicArray< Texture > & textures,
            Vector2u windowBounds,
            int level,
            float playerExp,
            float playerExpNext,
            int playerHp,
            int playerHpMax,
            int playerScore,
            int upKey,
            int downKey,
            int leftKey,
            int rightKey,
            int fireKey,
            int typeOfBullet)
```

Create player.

Check if the texture array is empty

Create a new player with the specified parameters.

This function creates a new player instance using the provided parameters, including textures, window bounds, player statistics, and control keys.

Parameters

textures	The array of textures to be used by the player.
windowBounds	The bounds of the window in which the player will operate.
level	The level of the player.

Parameters

playerExp	The experience points of the player.
playerExpNext	The experience points required for the next level.
playerHp	The current health points of the player.
playerHpMax	The maximum health points of the player.
playerScore	The score of the player.
upKey	The key used to move up.
downKey	The key used to move down.
leftKey	The key used to move left.
rightKey	The key used to move right.
fireKey	The key used to fire.
typeOfBullet	The type of bullet the player uses.

Returns

Player∗ A pointer to the newly created player instance.

Exceptions

std::invalid_argument	If the texture array is empty.

5.7.2.2 exit()

```
void Game::exit ()
```

Exit the game.

This function closes the game window and saves the current game data for the number of players.

Exceptions

std::runtime_error	If saving data fails.

Close the game window

Save game data based on the number of players

5.7.2.3 getGameState()

```
void Game::getGameState (
            int & numberOfPlayers,
            int & player1Level,
            int & player1Hp,
            int & player1HpMax,
            float & player1Exp,
            float & player1ExpNext,
            int & player1Score,
```

```
int & player2Level,
int & player2Hp,
int & player2HpMax,
float & player2Exp,
float & player2ExpNext,
int & player2Score,
bool & upperWeapon,
int & upperWeaponLevel,
bool & lowerWeapon,
int & lowerWeaponLevel)
```

Get the current game state including player information and weapon states.

This function sets the provided references with the current game state information. If the number of players is one, it throws an exception if player 2 is accessed.

Parameters

numberOfPlayers	The reference to store the number of players.
player1Level	The reference to store player 1's level.
player1Hp	The reference to store player 1's current health points.
player1HpMax	The reference to store player 1's maximum health points.
player1Exp	The reference to store player 1's experience points.
player1ExpNext	The reference to store player 1's experience points needed for the next level.
player1Score	The reference to store player 1's score.
player2Level	The reference to store player 2's level.
player2Hp	The reference to store player 2's current health points.
player2HpMax	The reference to store player 2's maximum health points.
player2Exp	The reference to store player 2's experience points.
player2ExpNext	The reference to store player 2's experience points needed for the next level.
player2Score	The reference to store player 2's score.
upperWeapon	The reference to store the state of the upper weapon.
upperWeaponLevel	The reference to store the level of the upper weapon.
lowerWeapon	The reference to store the state of the lower weapon.
lowerWeaponLevel	The reference to store the level of the lower weapon.

Exceptions

std::runtime_error	If player 1 or player 2 does not exist when expected.

Set the number of players

Get player 1 state

Get player 2 state if there are two players

Get upper weapon state

Get lower weapon state

5.7.2.4 initUI()

```
void Game::initUI ()
```

Initialize the UI elements.

This function loads the font, sets up text elements, progress bars, and initializes buttons for restarting, exiting, and pausing the game.

Exceptions

std::runtime_error	If the font fails to load.

Load font for UI elements

Set font for text elements

Set character size and color for Follow Player Text

Set character size and color for Stationary Text

Set size, color, and outline for Player Exp Bar

Set character size and color for Player Level Text

Create buttons

Initialize Restart Button

Initialize Exit Button

Initialize Pause Button

Load font for game over text

Set font, size, color, and position for Game Over Text

5.7.2.5 render()

```
void Game::render ()
```

Renders the game elements.

This method renders the game elements, including players, enemies, and UI components. It checks for null pointers and handles exceptions appropriately. Check if the window pointer is null

Check if the game is not over

Render players

Render enemies

Render UI elements

Handle any exceptions and print error messages

5.7.2.6 renderUI()

```
void Game::renderUI ()
```

Render UI elements.

This function renders all UI elements including text tags and buttons. Check if the window pointer is null

Check if the game is not over

Render UI elements for each player

Render text tags

Check if any button pointer is null

Render buttons

Handle any exceptions and print error messages

5.7.2.7 restart()

```
void Game::restart ()
```

Restart the game.

This function clears all current players and enemies, resets the game state, reloads saved data, and recreates players. It also decrements the times counter and emits a signal indicating the game reset.

Exceptions

std::runtime_error	If loading saved data fails.

Delete existing player objects

Delete existing enemy objects

Reset game over state and position of game over text

Save default data and load saved data

Reset player count

Create players

Decrement times and set game started state

Emit signal indicating game reset

5.7.2.8 update()

```
void Game::update (
            const float & dt)
```

Update the game state.

This function handles the game logic, including player updates, enemy spawning and updates, collision detection, and UI updates. It also manages the game over state and button interactions.

Parameters

dt	Delta time since the last frame update.

Exceptions

std::runtime_error	If there is an error in loading saved data or updating button states.

Check if this is the first update

Load saved data and create players

Delete previous players

Reset player count

Create new players

Update game logic only if the game is not paused

Increment enemy spawn timer

Spawn new enemies if the spawn timer exceeds the maximum value

Function to generate random numbers with a 50/50 chance

Spawn MoveLeftEnemy with a 50% chance

Randomize properties for MoveLeftEnemy

Spawn FollowEnemy with a 50% chance

Randomize properties for FollowEnemy

Reset enemy spawn timer

Update players

Update enemies

Default behavior if player to follow is invalid

Remove enemy if out of window bounds

Continue to the next iteration to avoid accessing invalid memory

Check enemy collision with players

Update text tags

Move game over text

Update buttons

Update button states based on mouse position

Set button click actions

5.7.2.9 updateUI()

```
void Game::updateUI (
            int index)
```

Update the UI elements for a specified player.

This function updates the position and content of the UI elements (such as text and experience bar) to reflect the current state of the specified player.

Parameters

index	The index of the player whose UI elements are to be updated.

Exceptions

std::out_of_range	If the player index is out of bounds.

Assert that the player index is within bounds

Update UI elements for the specified player

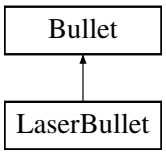
Exp Bar

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Game.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Game.cpp

5.8 LaserBullet Class Reference

Inheritance diagram for LaserBullet:



Public Member Functions

- LaserBullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new LaserBullet object.
- void Update (const float &dt) override

  Updates the LaserBullet's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture * **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.8.1 Constructor & Destructor Documentation

5.8.1.1 LaserBullet()

```
LaserBullet::LaserBullet (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new LaserBullet object.

Parameters

pos	Initial position of the LaserBullet.
level	Level of the LaserBullet.
direction	Direction of the LaserBullet.
initialVelocity	Initial velocity of the LaserBullet.
maxVelocity	Maximum velocity of the LaserBullet.
acceleration	Acceleration of the LaserBullet.

5.8.2 Member Function Documentation

5.8.2.1 Update()

```
void LaserBullet::Update (
            const float & dt)  [override], [virtual]
```

Updates the LaserBullet's movement.

Parameters

dt	Time elapsed since the last update.

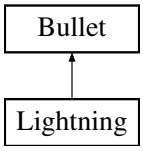
Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/LaserBullet.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/LaserBullet.cpp

5.9 Lightning Class Reference

Inheritance diagram for Lightning:



Public Member Functions

- Lightning (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Lightning object.
- void Update (const float &dt) override

  Updates the Lightning's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.9.1 Constructor & Destructor Documentation

5.9.1.1 Lightning()

```
Lightning::Lightning (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new Lightning object.

Parameters

pos	Initial position of the Lightning.
level	Level of the Lightning.
direction	Direction of the Lightning.
initialVelocity	Initial velocity of the Lightning.
maxVelocity	Maximum velocity of the Lightning.
acceleration	Acceleration of the Lightning.

5.9.2 Member Function Documentation

5.9.2.1 Update()

```
void Lightning::Update (
            const float & dt)  [override], [virtual]
```

Updates the Lightning's movement.

Parameters

dt	Time elapsed since the last update.

Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Lightning.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Lightning.cpp

5.10 Menu Class Reference

Public Member Functions

- Menu (sf::RenderWindow *window, Game *game)

  Construct a new Menu::Menu object.
- void render (sf::RenderTarget &target)

  Render the menu on the given render target.
- void update (const sf::Vector2f &mousePos, sf::Event &event)

  Update the menu based on mouse position and events.
- void addButton (float x, float y, float width, float height, float cornerRadius, sf::Font &font, const std::string &text)

  Add a button to the main menu.
- void addButtonToBeginMenu (float x, float y, float width, float height, float cornerRadius, sf::Font &font, const std::string &text)

  Add a button to the beginning menu.
- void resetColor ()

  Reset color for specific buttons related to weapon types.
- void resetUpperLevelColor ()

  Reset color for upper level buttons.
- void resetLowerLevelColor ()

  Reset color for lower level buttons.
- void resetAllColor ()

  Reset color for all buttons.
- void **setNewGame** (bool newGame)
- bool isButtonPressed (int index) const

  Check if a button at the given index is pressed.

5.10.1 Constructor & Destructor Documentation

5.10.1.1 Menu()

```
Menu::Menu (
            sf::RenderWindow * window,
            Game * game)
```

Construct a new Menu::Menu object.

Parameters

window	Pointer to the SFML RenderWindow
game	Pointer to the Game object

Load font for menu text

Set up title text for the menu

Add buttons to the beginning menu

Add buttons to the main menu

5.10.2 Member Function Documentation

5.10.2.1 addButton()

```
void Menu::addButton (
            float x,
            float y,
            float width,
            float height,
            float cornerRadius,
            sf::Font & font,
            const std::string & text)
```

Add a button to the main menu.

Parameters

x	X position of the button
y	Y position of the button
width	Width of the button
height	Height of the button
cornerRadius	Corner radius of the button
font	Font used for the button text
text	Text displayed on the button

5.10.2.2 addButtonToBeginMenu()

```
void Menu::addButtonToBeginMenu (
            float x,
            float y,
            float width,
            float height,
            float cornerRadius,
            sf::Font & font,
            const std::string & text)
```

Add a button to the beginning menu.

Parameters

x	X position of the button
y	Y position of the button
width	Width of the button
height	Height of the button
cornerRadius	Corner radius of the button
font	Font used for the button text
text	Text displayed on the button

5.10.2.3 isButtonPressed()

```
bool Menu::isButtonPressed (
            int index) const
```

Check if a button at the given index is pressed.

Parameters

index	Index of the button

Returns

true if the button is pressed, false otherwise

5.10.2.4 render()

```
void Menu::render (
            sf::RenderTarget & target)
```

Render the menu on the given render target.

Parameters

target	Render target to draw on

Clear the window

Render the appropriate menu based on the state

Render buttons for the beginning menu

Render title and buttons for the main menu

Display the window

5.10.2.5 resetAllColor()

```
void Menu::resetAllColor ()
```

Reset color for all buttons.

Reset color for all buttons

5.10.2.6 resetColor()

```
void Menu::resetColor ()
```

Reset color for specific buttons related to weapon types.

Reset color for specific buttons

5.10.2.7 resetLowerLevelColor()

```
void Menu::resetLowerLevelColor ()
```

Reset color for lower level buttons.

Reset color for lower level buttons

5.10.2.8 resetUpperLevelColor()

```
void Menu::resetUpperLevelColor ()
```

Reset color for upper level buttons.

Reset color for upper level buttons

5.10.2.9 update()

```
void Menu::update (
            const sf::Vector2f & mousePos,
            sf::Event & event)
```

Update the menu based on mouse position and events.

Parameters

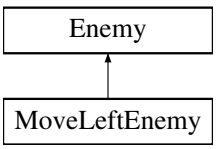
mousePos	Position of the mouse cursor
event	SFML event to process

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Menu.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Menu.cpp

5.11 MoveLeftEnemy Class Reference

Inheritance diagram for MoveLeftEnemy:



Public Member Functions

- MoveLeftEnemy (Texture *texture, Vector2u windowBounds, Vector2f position, Vector2f direction, Vector2f scale, float exp, int hpMax, int damageMax, int damageMin)

  Constructs a new MoveLeftEnemy object.
- void Move (float dt, Vector2f playerPosition) override

  Moves the MoveLeftEnemy.

Public Member Functions inherited from Enemy

- Enemy (sf::Texture *texture, sf::Vector2u windowBounds, sf::Vector2f position, sf::Vector2f direction, sf::↩ Vector2f scale, float exp, int hpMax, int damageMax, int damageMin)

  Constructs a new Enemy object.
- virtual ~**Enemy** ()

  Destroys the Enemy object.
- const int **getDamage** () const
- const int **getHp** () const
- const int **getHpMax** () const
- const float **getExp** () const
- const bool **isDead** () const
- const sf::FloatRect **getBounds** () const
- const sf::Vector2f & **getPosition** () const
- const int **getPlayerToFollow** () const
- void TakeDamage (int damage)

  Inflicts damage on the enemy.
- virtual void Update (float dt, Vector2f playerPosition)

  Updates the state of the enemy.
- void Render (sf::RenderTarget &target)

  Renders the enemy on the screen.

Additional Inherited Members

Protected Attributes inherited from Enemy

- float **dtMultiplier**
- sf::Sprite **sprite**
- sf::Texture * **texture**
- sf::Vector2f **direction**
- sf::Vector2u **windowBounds**
- float **damageTimer**
- float **damageTimerMax**
- int **hp**
- int **hpMax**
- int **damageMin**
- int **damageMax**
- float **exp**
- int **enemySpawnTimer**
- int **enemySpawnTimerMax**
- int **playerToFollow** = 0
- float **speedx**
- float **speedy**

5.11.1 Constructor & Destructor Documentation

5.11.1.1 MoveLeftEnemy()

```
MoveLeftEnemy::MoveLeftEnemy (
            Texture * texture,
            Vector2u windowBounds,
            Vector2f position,
            Vector2f direction,
            Vector2f scale,
            float exp,
            int hpMax,
            int damageMax,
            int damageMin)
```

Constructs a new MoveLeftEnemy object.

Parameters

texture	Pointer to the texture for the enemy.
windowBounds	The bounds of the game window.
position	Initial position of the enemy.
direction	Initial movement direction of the enemy.
scale	Scale factor for the enemy sprite.
exp	Experience points gained when defeating the enemy.
hpMax	Maximum hit points of the enemy.
damageMax	Maximum damage inflicted by the enemy.
damageMin	Minimum damage inflicted by the enemy.

5.11.2 Member Function Documentation

5.11.2.1 Move()

```
void MoveLeftEnemy::Move (
            float dt,
            Vector2f playerPosition)  [override], [virtual]
```

Moves the MoveLeftEnemy.

Parameters

dt	Time elapsed since the last update.
playerPosition	Position of the player.

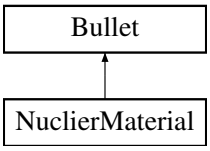
Implements Enemy.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/MoveLeft.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Enemys/MoveLeft.cpp

5.12 NuclierMaterial Class Reference

Inheritance diagram for NuclierMaterial:



Public Member Functions

- NuclierMaterial (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new NuclierMaterial object.
- void Update (const float &dt) override

  Updates the NuclierMaterial's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.12.1 Constructor & Destructor Documentation

5.12.1.1 NuclierMaterial()

```
NuclierMaterial::NuclierMaterial (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new NuclierMaterial object.

Parameters

pos	Initial position of the NuclierMaterial.
level	Level of the NuclierMaterial.
direction	Direction of the NuclierMaterial.
initialVelocity	Initial velocity of the NuclierMaterial.
maxVelocity	Maximum velocity of the NuclierMaterial.
acceleration	Acceleration of the NuclierMaterial.

5.12.2 Member Function Documentation

5.12.2.1 Update()

```
void NuclierMaterial::Update (
            const float & dt)  [override], [virtual]
```

Updates the NuclierMaterial's movement.

Parameters

dt	Time elapsed since the last update.

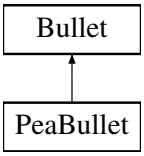
Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/NuclierMaterial.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/NuclierMaterial.cpp

5.13 PeaBullet Class Reference

Inheritance diagram for PeaBullet:



Public Member Functions

- PeaBullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=5.f, float max↩ Velocity=10.f, float acceleration=1.f)

  Constructs a new PeaBullet object.
- void Update (const float &dt) override

  Updates the PeaBullet's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.13.1 Constructor & Destructor Documentation

5.13.1.1 PeaBullet()

```
PeaBullet::PeaBullet (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 5.f,
            float maxVelocity = 10.f,
            float acceleration = 1.f)
```

Constructs a new PeaBullet object.

Parameters

pos	Initial position of the PeaBullet.
level	Level of the PeaBullet.
direction	Direction of the PeaBullet.
initialVelocity	Initial velocity of the PeaBullet.
maxVelocity	Maximum velocity of the PeaBullet.
acceleration	Acceleration of the PeaBullet.

5.13.2 Member Function Documentation

5.13.2.1 Update()

```
void PeaBullet::Update (
            const float & dt)  [override], [virtual]
```

Updates the PeaBullet's movement.

Parameters

dt	Time elapsed since the last update.

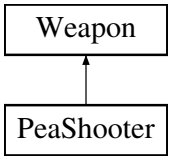
Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PeaBullet.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/PeaBullet.cpp

5.14 PeaShooter Class Reference

Inheritance diagram for PeaShooter:



Public Member Functions

- **PeaShooter** ()

  Default constructor for the PeaShooter class.
- PeaShooter (int level, int UpOrDown)

  Parameterized constructor for the PeaShooter class.
- virtual ~PeaShooter ()

  Destructor for the PeaShooter class.
- void Update (Vector2f targetPosition, FloatRect bounds, const float &dt) override

  Update method for the PeaShooter class.
- void Fire (const Vector2f &position, const Vector2f &direction, const Vector2f &scale) override

  Fire method for the PeaShooter class.
- void Upgrade () override

  Upgrade method for the PeaShooter class.

Public Member Functions inherited from Weapon

- Weapon (int type, int level, int UpOrDown, int fireRateMax=25, int damageMax=1)

  Parameterized constructor for the Weapon class.
- std::vector< Bullet * > & **getBullets** ()
- const Vector2f & **getPosition** () const
- const FloatRect **getBounds** () const
- void **setPosition** (const Vector2f &position)
- void Render (RenderTarget &target)

  Render method for the Weapon class.

Additional Inherited Members

Protected Attributes inherited from Weapon

- float **dtMultiplier**
- int **fireRate**
- int **fireRateMax**
- Sprite **sprite**
- Texture * **weaponTexture**
- std::vector< Bullet * > **bullets**
- int **bulletLevel**
- int **damage**
- int **damageMax**
- int **UpOrDown**
- int **level**

5.14.1 Constructor & Destructor Documentation

5.14.1.1 PeaShooter()

```
PeaShooter::PeaShooter (
            int level,
            int UpOrDown)
```

Parameterized constructor for the PeaShooter class.

Parameters

level	The level of the pea shooter.
UpOrDown	Whether the pea shooter is positioned up or down.

5.14.1.2 ~PeaShooter()

```
PeaShooter::~PeaShooter () [virtual]
```

Destructor for the PeaShooter class.

Destructor implementation, if needed

5.14.2 Member Function Documentation

5.14.2.1 Fire()

```
void PeaShooter::Fire (
            const Vector2f & position,
            const Vector2f & direction,
            const Vector2f & scale)  [override], [virtual]
```

Fire method for the PeaShooter class.

Parameters

position	The position to fire from.
direction	The firing direction.
scale	The scale of the fired object.

Reimplemented from Weapon.

5.14.2.2 Update()

```
void PeaShooter::Update (
            Vector2f targetPosition,
            FloatRect bounds,
            const float & dt)  [override], [virtual]
```

Update method for the PeaShooter class.

Parameters

targetPosition	The position of the target.
bounds	The bounds of the pea shooter.
dt	The delta time for updating.

Reimplemented from Weapon.

5.14.2.3 Upgrade()

```
void PeaShooter::Upgrade ()  [override], [virtual]
```

Upgrade method for the PeaShooter class.

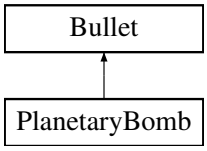
Reimplemented from Weapon.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/PeaShooter.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/PeaShooter.cpp

5.15 PlanetaryBomb Class Reference

Inheritance diagram for PlanetaryBomb:



Public Member Functions

- PlanetaryBomb (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float maxVelocity=50.f, float acceleration=1.f)

  Constructs a new PlanetaryBomb object.
- void Update (const float &dt) override

  Updates the PlanetaryBomb's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.15.1 Constructor & Destructor Documentation

5.15.1.1 PlanetaryBomb()

```
PlanetaryBomb::PlanetaryBomb (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new PlanetaryBomb object.

Parameters

pos	Initial position of the PlanetaryBomb.
level	Level of the PlanetaryBomb.
direction	Direction of the PlanetaryBomb.
initialVelocity	Initial velocity of the PlanetaryBomb.
maxVelocity	Maximum velocity of the PlanetaryBomb.
acceleration	Acceleration of the PlanetaryBomb.

5.15.2 Member Function Documentation

5.15.2.1 Update()

```
void PlanetaryBomb::Update (
            const float & dt)  [override], [virtual]
```

Updates the PlanetaryBomb's movement.

Parameters

dt	Time elapsed since the last update.

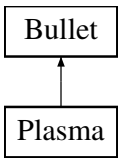
Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PlanetaryBomb.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/PlanetaryBomb.cpp

5.16 Plasma Class Reference

Inheritance diagram for Plasma:



Public Member Functions

- Plasma (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Plasma object.
- void Update (const float &dt) override

  Updates the Plasma bullet's movement.

Public Member Functions inherited from Bullet

- Bullet (Vector2f pos, int level, Vector2f direction=Vector2f(1.f, 0.f), float initialVelocity=2.f, float max↩ Velocity=50.f, float acceleration=1.f)

  Constructs a new Bullet object.
- const FloatRect **getBounds** () const
- const Vector2f & **getPosition** () const
- const int & **getDamage** () const
- void setTexture (int type, int level)

  Sets the texture of the bullet based on its type and level.
- void setDamage (int damage, int level)

  Sets the damage of the bullet based on its type and level.
- virtual void Move (const float &dt)

  Moves the bullet based on its current velocity.
- void Render (RenderTarget &target)

  Renders the bullet on the target.

Additional Inherited Members

Protected Attributes inherited from Bullet

- float **dtMultiplier**
- Sprite **sprite**
- Texture ∗ **texture**
- Vector2f **currentVelocity**
- Vector2f **direction**
- float **maxVelocity**
- float **acceleration**
- int **damage**
- int **damageMax**
- int **level**

5.16.1 Constructor & Destructor Documentation

5.16.1.1 Plasma()

```
Plasma::Plasma (
            Vector2f pos,
            int level,
            Vector2f direction = Vector2f(1.f, 0.f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Constructs a new Plasma object.

Parameters

pos	Initial position of the Plasma bullet.
level	Level of the Plasma bullet.
direction	Direction of the Plasma bullet.
initialVelocity	Initial velocity of the Plasma bullet.
maxVelocity	Maximum velocity of the Plasma bullet.
acceleration	Acceleration of the Plasma bullet.

5.16.2 Member Function Documentation

5.16.2.1 Update()

```
void Plasma::Update (
            const float & dt)  [override], [virtual]
```

Updates the Plasma bullet's movement.

Parameters

dt	Time elapsed since the last update.

Reimplemented from Bullet.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Plasma.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Plasma.cpp

5.17 Player Class Reference

Public Member Functions

- Player (DynamicArray< Texture > &textures, Vector2u windowBounds, int typeOfBullet, int UP=22, int DOWN=18, int LEFT=0, int RIGHT=3, int FIRE=57, int playerLevel=0, float playerExp=0, float playerExp↩Next=16, int playerHp=10, int playerHpMax=10, int playerScore=0, int upperWeaponLevel=0, int lower↩WeaponLevel=0, bool uperWeapon=false, bool lowerWeapon=false)

  Constructs a Player object.
- ~**Player** ()

  Destructor for Player.
- Bullet & getBullet (unsigned index)

  Retrieves a bullet by index.
- void removeBullet (unsigned index)

  Removes a bullet from the player's inventory.
- const int **getBulletSize** ()
- const Vector2f & **getPosition** () const
- DynamicArray< Weapon * > & **getWeapons** ()
- const String **getHpS** () const

- const FloatRect **getBounds** () const
- const bool **isDead** () const
- const int **getPlayerNumber** () const
- const int **getLevel** () const
- const int **getMaxLevel** () const
- const bool **getIsAtMaxLevel** () const
- const int **getExp** () const
- const int **getExpNext** () const
- const int **getStatPoints** () const
- const int **getHp** () const
- const int **getHpMax** () const
- const int **getScore** () const
- const int **getCurrentWeapon** () const
- bool **gainExp** (float exp)
- template<typename T >
  void setBulletType (Vector2f pos, int upgrade, int level, Vector2f directionUp=Vector2f(1.f, -0.1f), Vector2f directionDown=Vector2f(1.f, 0.1f), float initialVelocity=2.f, float maxVelocity=50.f, float acceleration=1.f)

  Sets the type of bullet fired by the player.
- void **setToDefault** ()
- void **setCurrentWeapon** (int weapon)
- void **setUpperWeaponLevel** (int level)
- void **setLowerWeaponLevel** (int level)
- void **setUperWeapon** (bool uperWeapon)
- void **setLowerWeapon** (bool lowerWeapon)
- void **setLevel** (int level)
- void **setExp** (float exp)
- void **setExpNext** (float expNext)
- void **setStatPoints** (int statPoints)
- void **setHp** (int hp)
- void **setHpMax** (int hpMax)
- void **setScore** (int score)
- void Move (const float &dt)

  Moves the player based on input and physics.
- bool updateLevelingSystem ()

  Updates the player's leveling system.
- void **LevelUp** ()

  Levels up the player.
- void addWeapon (int type, int UpOrDown, int level)

  Adds a weapon to the player's inventory.
- void CombatUpdate ()

  Updates combat-related actions of the player.
- void UpdateAccessories (const float &dt)

  Updates the accessories (weapons) of the player.
- void TakeDamage (int damage)

  Inflicts damage on the player.
- void Update (Vector2u windowBounds, const float &dt)

  Updates the player's state.
- void Render (RenderTarget &target)

  Renders the player and associated elements.

Static Public Attributes

- static unsigned **players** = 0

5.17.1 Constructor & Destructor Documentation

5.17.1.1 Player()

```
Player::Player (
            DynamicArray< Texture > & textures,
            Vector2u windowBounds,
            int typeOfBullet,
            int UP = 22,
            int DOWN = 18,
            int LEFT = 0,
            int RIGHT = 3,
            int FIRE = 57,
            int playerLevel = 0,
            float playerExp = 0,
            float playerExpNext = 16,
            int playerHp = 10,
            int playerHpMax = 10,
            int playerScore = 0,
            int upperWeaponLevel = 0,
            int lowerWeaponLevel = 0,
            bool uperWeapon = false,
            bool lowerWeapon = false)
```

Constructs a Player object.

Parameters

textures	Reference to a DynamicArray of textures.
windowBounds	The bounds of the game window.
typeOfBullet	The type of bullet the player uses.
UP	The keycode for moving up.
DOWN	The keycode for moving down.
LEFT	The keycode for moving left.
RIGHT	The keycode for moving right.
FIRE	The keycode for firing.
playerLevel	The player's level.
playerExp	The player's experience points.
playerExpNext	The experience points needed for the next level.
playerHp	The player's health points.
playerHpMax	The maximum health points of the player.
playerScore	The player's score.
upperWeaponLevel	The level of the upper weapon.
lowerWeaponLevel	The level of the lower weapon.
upperWeapon	Flag indicating whether the upper weapon is equipped.
lowerWeapon	Flag indicating whether the lower weapon is equipped.

Delta time multiplier

Stats

Set up the sprite

Controls

Player number

5.17.2 Member Function Documentation

5.17.2.1 addWeapon()

```
void Player::addWeapon (
            int type,
            int UpOrDown,
            int level)
```

Adds a weapon to the player's inventory.

Parameters

type	The type of weapon to add.
UpOrDown	Specifies upper or lower weapon.
level	The level of the weapon.

5.17.2.2 CombatUpdate()

```
void Player::CombatUpdate ()
```

Updates combat-related actions of the player.

change main gun level with number keys

Fire main gun

Fire side guns

5.17.2.3 getBullet()

```
Bullet & Player::getBullet (
            unsigned index)
```

Retrieves a bullet by index.

Parameters

index	The index of the bullet.

Returns

Reference to the bullet.

Exceptions

An	exception if index is out of bounds.

5.17.2.4 Move()

```
void Player::Move (
            const float & dt)
```

Moves the player based on input and physics.

Parameters

dt	The change in time since the last update.

Update velocity

Drag

Wall collision

5.17.2.5 removeBullet()

```
void Player::removeBullet (
            unsigned index)
```

Removes a bullet from the player's inventory.

Parameters

index	The index of the bullet to remove.

Exceptions

An	exception if index is out of bounds.

5.17.2.6 Render()

```
void Player::Render (
            RenderTarget & target)
```

Renders the player and associated elements.

Parameters

target	The render target.

Render bullets

Render weapons

Render the player

5.17.2.7 setBulletType()

```
template<typename T >
void Player::setBulletType (
            Vector2f pos,
            int upgrade,
            int level,
            Vector2f directionUp = Vector2f(1.f, -0.1f),
            Vector2f directionDown = Vector2f(1.f, 0.1f),
            float initialVelocity = 2.f,
            float maxVelocity = 50.f,
            float acceleration = 1.f)
```

Sets the type of bullet fired by the player.

Template Parameters

T	The type of bullet.

Parameters

pos	The position from which the bullet is fired.
upgrade	The upgrade level of the bullet.
level	The level of the player.
directionUp	The direction vector for upper bullet.
directionDown	The direction vector for lower bullet.
initialVelocity	The initial velocity of the bullet.
maxVelocity	The maximum velocity of the bullet.
acceleration	The acceleration of the bullet.

5.17.2.8 TakeDamage()

```
void Player::TakeDamage (
            int damage)
```

Inflicts damage on the player.

Parameters

damage	The amount of damage to inflict.

5.17.2.9 Update()

```
void Player::Update (
            Vector2u windowBounds,
            const float & dt)
```

Updates the player's state.

Parameters

windowBounds	The bounds of the game window.
dt	The change in time since the last update.

Update fire rate

Update weapons and bullets

Remove bullets that have gone off-screen

Decrement j to account for the erased element

Update player movement and accessories

Check for firing input and perform combat update

5.17.2.10 UpdateAccessories()

```
void Player::UpdateAccessories (
            const float & dt)
```

Updates the accessories (weapons) of the player.

Parameters

dt	The change in time since the last update.

5.17.2.11 updateLevelingSystem()

```
bool Player::updateLevelingSystem ()
```

Updates the player's leveling system.

Returns

True if player leveled up, false otherwise.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Player.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Player.cpp

5.18 TextTag Class Reference

Public Member Functions

- TextTag (Font ∗font, std::string text, Vector2f position, Vector2f direction, unsigned int size, float timerMax, bool accelerate, Color color)

  Constructs a TextTag object.
- TextTag (const TextTag &other)

  Copy constructor for TextTag.
- TextTag & operator= (const TextTag &other)

  Assignment operator for TextTag.
- const float & **getTimer** () const
- void Update (const float &dt)

  Updates the state of the TextTag.
- void Render (RenderTarget &target)

  Renders the TextTag on a render target.

5.18.1 Constructor & Destructor Documentation

5.18.1.1 TextTag() [1/2]

```
TextTag::TextTag (
            Font * font,
            std::string text,
            Vector2f position,
            Vector2f direction,
            unsigned int size,
            float timerMax,
            bool accelerate,
            Color color)
```

Constructs a TextTag object.

Parameters

font	Pointer to the font used for the text tag.
text	The string to be displayed as the text tag.
position	The initial position of the text tag.
direction	The direction in which the text tag moves.
size	The character size of the text tag.
timerMax	The maximum lifetime of the text tag.
accelerate	Flag indicating whether the text tag should accelerate as it ages.
color	The color of the text tag.

5.18.1.2 TextTag() [2/2]

```
TextTag::TextTag (
            const TextTag & other)
```

Copy constructor for TextTag.

Parameters

other	The TextTag object to be copied.

5.18.2 Member Function Documentation

5.18.2.1 operator=()

```
TextTag & TextTag::operator= (
            const TextTag & other)
```

Assignment operator for TextTag.

Parameters

other	The TextTag object to be assigned.

Returns

Reference to the assigned TextTag object.

5.18.2.2 Render()

```
void TextTag::Render (
            RenderTarget & target)
```

Renders the TextTag on a render target.

Parameters

target	The render target on which to draw the TextTag.

5.18.2.3 Update()

```
void TextTag::Update (
            const float & dt)
```

Updates the state of the TextTag.

Parameters

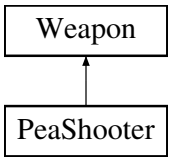
dt	The change in time since the last update.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/TextTag.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/TextTag.cpp

5.19 Weapon Class Reference

Inheritance diagram for Weapon:



Public Member Functions

- Weapon (int type, int level, int UpOrDown, int fireRateMax=25, int damageMax=1)

  Parameterized constructor for the Weapon class.
- std::vector< Bullet * > & **getBullets** ()
- const Vector2f & **getPosition** () const
- const FloatRect **getBounds** () const
- void **setPosition** (const Vector2f &position)
- virtual void Update (Vector2f targetPosition, FloatRect bounds, const float &dt)

  Update method for the Weapon class.
- virtual void Fire (const Vector2f &position, const Vector2f &direction, const Vector2f &scale)

  Fire method for the Weapon class.
- virtual void Upgrade ()

  Upgrade method for the Weapon class.
- void Render (RenderTarget &target)

  Render method for the Weapon class.

Protected Attributes

- float **dtMultiplier**
- int **fireRate**
- int **fireRateMax**
- Sprite **sprite**
- Texture ∗ **weaponTexture**
- std::vector< Bullet ∗ > **bullets**
- int **bulletLevel**
- int **damage**
- int **damageMax**
- int **UpOrDown**
- int **level**

5.19.1 Constructor & Destructor Documentation

5.19.1.1 Weapon()

```
Weapon::Weapon (
            int type,
            int level,
            int UpOrDown,
            int fireRateMax = 25,
            int damageMax = 1)
```

Parameterized constructor for the Weapon class.

Parameters

type	The type of weapon.
level	The level of the weapon.
UpOrDown	Whether the weapon is positioned up or down.
fireRateMax	The maximum fire rate of the weapon.
damageMax	The maximum damage of the weapon.

5.19.2 Member Function Documentation

5.19.2.1 Fire()

```
void Weapon::Fire (
            const Vector2f & position,
            const Vector2f & direction,
            const Vector2f & scale)  [virtual]
```

Fire method for the Weapon class.

Parameters

position	The position to fire from.
direction	The firing direction.
scale	The scale of the fired object.

Reimplemented in PeaShooter.

5.19.2.2 Render()

```
void Weapon::Render (
            RenderTarget & target)
```

Render method for the Weapon class.

Renders the weapon and its bullets.

Parameters

target	The render target.

Render bullets

Render the weapon

5.19.2.3 Update()

```
void Weapon::Update (
            Vector2f targetPosition,
            FloatRect bounds,
            const float & dt)  [virtual]
```

Update method for the Weapon class.

Parameters

targetPosition	The position of the target.
bounds	The bounds of the weapon.
dt	The delta time for updating.

Reimplemented in PeaShooter.

5.19.2.4 Upgrade()

```
void Weapon::Upgrade ()  [virtual]
```

Upgrade method for the Weapon class.

Increases the damage and reduces the fire rate.

Reimplemented in PeaShooter.

The documentation for this class was generated from the following files:

- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/Weapon.hpp
- /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Weapons/Weapon.cpp

Chapter 6

File Documentation

6.1 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/Bullet.hpp File Reference

This is the header file for the Bullet class, which is used to create bullets for the player to shoot.

```
#include "../Resources/BulletResources.hpp"
#include "../TextTag.hpp"
```

Classes

- class Bullet

6.1.1 Detailed Description

This is the header file for the Bullet class, which is used to create bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.2 Bullet.hpp

Go to the documentation of this file.

```
#pragma once

#include "../Resources/BulletResources.hpp"
#include "../TextTag.hpp"

class Bullet
{
protected:
    float dtMultiplier;

    Sprite sprite;
    Texture *texture;
    Vector2f currentVelocity;
    Vector2f direction;
    float maxVelocity;
    float acceleration;

    int damage;
    int damageMax;

    int level;

public:
    Bullet(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
                float initialVelocity = 2.f, float maxVelocity = 50.f,
                float acceleration = 1.f);
    virtual ~Bullet();

    // Accessors
    const FloatRect getBounds() const { return this->sprite.getGlobalBounds(); }
    const Vector2f &getPosition() const { return this->sprite.getPosition(); }
    const int &getDamage() const { return this->damage; }

    // Setters
    void setTexture(int type, int level);
    void setDamage(int damage, int level);

    // Functions

    virtual void Move(const float &dt);
    virtual void Update(const float &dt);
    void Render(RenderTarget &target);
};
```

6.3 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/DarkMatter.hpp File Reference

This is the header file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class DarkMatter

6.3.1 Detailed Description

This is the header file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.4 DarkMatter.hpp

Go to the documentation of this file.

```
#pragma once

#include "Bullet.hpp"

class DarkMatter : public Bullet {
public:
    DarkMatter(Vector2f pos, int level, Vector2f direction = Vector2f(1.f, 0.f),
                float initialVelocity = 2.f, float maxVelocity = 50.f,
                float acceleration = 1.f);

    void Update(const float &dt) override;
};
```

6.5 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/LaserBullet.hpp File Reference

This is the header file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class LaserBullet

6.5.1 Detailed Description

This is the header file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.6 LaserBullet.hpp

Go to the documentation of this file.

```
#pragma once

#include "Bullet.hpp"

class LaserBullet : public Bullet
{
public:
    LaserBullet(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
                float initialVelocity = 2.f, float maxVelocity = 50.f,
                float acceleration = 1.f);

    void Update(const float &dt) override;
};
```

6.7 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/Lightning.hpp File Reference

This is the header file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class Lightning

6.7.1 Detailed Description

This is the header file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.8 Lightning.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "Bullet.hpp"
00014
00015 class Lightning : public Bullet {
00016 public:
00017     Lightning(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
00018                 float initialVelocity = 2.f, float maxVelocity = 50.f,
00019                 float acceleration = 1.f);
00020
00021     void Update(const float &dt) override;
00022 };
```

6.9 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/NuclierMaterial.hpp File Reference

This is the header file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class NuclierMaterial

6.9.1 Detailed Description

This is the header file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.10 NuclierMaterial.hpp

Go to the documentation of this file.

```
#pragma once

#include "Bullet.hpp"

class NuclierMaterial : public Bullet {
public:
    NuclierMaterial(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
                float initialVelocity = 2.f, float maxVelocity = 50.f,
                float acceleration = 1.f);

    void Update(const float &dt) override;
};
```

6.11 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/PeaBullet.hpp File Reference

This is the header file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class PeaBullet

6.11.1 Detailed Description

This is the header file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.12 PeaBullet.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "Bullet.hpp"
00014
00015 class PeaBullet : public Bullet {
00016 public:
00017     PeaBullet(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
00018                 float initialVelocity = 5.f, float maxVelocity = 10.f,
00019                 float acceleration = 1.f);
00020
00021     void Update(const float &dt) override;
00022 };
```

6.13 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/PlanetaryBomb.hpp File Reference

This is the header file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class PlanetaryBomb

6.13.1 Detailed Description

This is the header file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.14 PlanetaryBomb.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "Bullet.hpp"
00014
00015 class PlanetaryBomb : public Bullet {
00016 public:
00017     PlanetaryBomb(Vector2f pos, int level,Vector2f direction = Vector2f(1.f, 0.f),
00018                   float initialVelocity = 2.f, float maxVelocity = 50.f,
00019                   float acceleration = 1.f);
00020
00021     void Update(const float &dt) override;
00022 };
```

6.15 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Bullets/Plasma.hpp File Reference

This is the header file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot.

```
#include "Bullet.hpp"
```

Classes

- class Plasma

6.15.1 Detailed Description

This is the header file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.16 Plasma.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "Bullet.hpp"
00014
00015 class Plasma : public Bullet {
00016 public:
00017     Plasma(Vector2f pos,int level, Vector2f direction = Vector2f(1.f, 0.f),
00018                 float initialVelocity = 2.f, float maxVelocity = 50.f,
00019                 float acceleration = 1.f);
00020
00021     void Update(const float &dt) override;
00022 };
```

6.17 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Enemys/Enemy.hpp File Reference

This is the header file for the Enemy class, which is used to create enemies for the player to fight against.

```
#include "../Bullets/Bullet.hpp"
#include "../Resources/Fonts.hpp"
#include "../Resources/Textures.hpp"
#include "../Player.hpp"
```

Classes

- class Enemy

Enumerations

- enum **enemyTypes** { **MOVE_LEFT** = 0 , **FOLLOW** , **FOLLOW_FAST** , **FOLLOW_SHOOT** }

6.17.1 Detailed Description

This is the header file for the Enemy class, which is used to create enemies for the player to fight against.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.18 Enemy.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "../Bullets/Bullet.hpp"
00014 #include "../Resources/Fonts.hpp"
00015 #include "../Resources/Textures.hpp"
00016 #include "../Player.hpp"
00017
00018 class Enemy
00019 {
00020 protected:
00021     float dtMultiplier;
00022     sf::Sprite sprite;
00023     sf::Texture *texture;
00024     sf::Vector2f direction;
00025     sf::Vector2u windowBounds;
00026
00027     float damageTimer;
00028     float damageTimerMax;
00029
00030     int hp;
00031     int hpMax;
00032     int damageMin;
00033     int damageMax;
00034
00035     float exp;
00036
00037     int enemySpawnTimer;
00038     int enemySpawnTimerMax;
00039
00040     int playerToFollow = 0;
00041
00042     float speedx;
00043     float speedy;
00044
00045 public:
00046     Enemy(sf::Texture *texture, sf::Vector2u windowBounds,
```

```
            sf::Vector2f position, sf::Vector2f direction,
            sf::Vector2f scale, float exp,
            int hpMax, int damageMax, int damageMin);
    virtual ~Enemy();

    // Accessors
    inline const int getDamage() const { return rand() % this->damageMax + this->damageMin; }
    inline const int getHp() const { return this->hp; }
    inline const int getHpMax() const { return this->hpMax; }
    inline const float getExp() const { return this->exp; }
    inline const bool isDead() const { return this->hp <= 0; }

    inline const sf::FloatRect getBounds() const { return this->sprite.getGlobalBounds(); }
    inline const sf::Vector2f &getPosition() const { return this->sprite.getPosition(); }
    inline const int getPlayerToFollow() const { return this->playerToFollow; }

    // Functions
    void TakeDamage(int damage);
    virtual void Move(float dt, Vector2f playerPosition) = 0;
    virtual void Update(float dt, Vector2f playerPosition);
    void Render(sf::RenderTarget &target);
};

enum enemyTypes
{
    MOVE_LEFT = 0,
    FOLLOW,
    FOLLOW_FAST,
    FOLLOW_SHOOT,
};
```

6.19 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Enemys/Follow.hpp File Reference

This is the header file for the FollowEnemy class, child of Enemy, which is used to create enemies that follow the player.

```
#include "./Enemy.hpp"
```

Classes

- class FollowEnemy

6.19.1 Detailed Description

This is the header file for the FollowEnemy class, child of Enemy, which is used to create enemies that follow the player.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.20 Follow.hpp

Go to the documentation of this file.

```
#pragma once

#include "./Enemy.hpp"

class FollowEnemy : public Enemy
{
private:
    float followTimer;
    float followTimerMax;
    bool following = true;
    Vector2f initialDirection;
    float initialDistance;
public:
    FollowEnemy(Texture *texture, Vector2u windowBounds,
                Vector2f position, Vector2f direction,
                Vector2f scale, float exp,
                int hpMax, int damageMax, int damageMin, int playerToFollow = 0);
    virtual ~FollowEnemy();

    void Move(float dt,  Vector2f playerPosition) override;


};
```

6.21 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Enemys/MoveLeft.hpp File Reference

This is the header file for the MoveLeftEnemy class, child of Enemy, which is used to create enemies that move left.

```
#include "./Enemy.hpp"
```

Classes

- class MoveLeftEnemy

6.21.1 Detailed Description

This is the header file for the MoveLeftEnemy class, child of Enemy, which is used to create enemies that move left.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.22 MoveLeft.hpp

Go to the documentation of this file.

```
#pragma once

#include "./Enemy.hpp"

class MoveLeftEnemy : public Enemy
{
private:


public:
    MoveLeftEnemy(Texture *texture, Vector2u windowBounds,
                  Vector2f position, Vector2f direction,
                  Vector2f scale, float exp,
                  int hpMax, int damageMax, int damageMin);
    virtual ~MoveLeftEnemy();

    void Move(float dt,  Vector2f playerPosition) override;
};
```

6.23 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Game.hpp File Reference

This is the header file for the Game class, which is used to create the game and manage the game state.

```
#include "Player.hpp"
#include "./Resources/Fonts.hpp"
#include "./Resources/Textures.hpp"
#include "./Enemys/Enemy.hpp"
#include "./Enemys/MoveLeft.hpp"
#include "./Enemys/Follow.hpp"
#include "../include/Weapons/PeaShooter.hpp"
#include "TextTag.hpp"
#include "./Resources/Button.hpp"
#include <iostream>
#include <vector>
```

Classes

- class Game

6.23.1 Detailed Description

This is the header file for the Game class, which is used to create the game and manage the game state.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.24 Game.hpp

Go to the documentation of this file.

```
#pragma once

// Include necessary headers
#include "Player.hpp"
#include "./Resources/Fonts.hpp"
#include "./Resources/Textures.hpp"
#include "./Enemys/Enemy.hpp"
#include "./Enemys/MoveLeft.hpp"
#include "./Enemys/Follow.hpp"
#include "../include/Weapons/PeaShooter.hpp"
#include "TextTag.hpp"
#include "./Resources/Button.hpp"
#include <iostream>
#include <vector>

class Game
{
private:
    // Private member variables
    RenderWindow *window;
    float dtMultiplier;
    bool gameOver = false;
    bool gamePaused = false;
    bool inGame = false;
    int timesPressed = 0;

    // UI elements
    Text followPlayerText;
    Text stationaryText;
    Text gameOverText;
    Text playerLevelText;
    DynamicArray<TextTag> textTags;

    // Fonts
    Font font;
    Font gameOverFont;

    // Bars
    RectangleShape playerHealthBar;
    RectangleShape playerExpBar;
    RectangleShape playerExpBarOutline;

    // Buttons
    Button *restartButton;
    Button *exitButton;
    Button *pauseButton;

    // Resources
    DynamicArray<Texture> textures;

    // Players
    std::vector<Player *> players;

    // Enemies
    DynamicArray<Enemy *> enemies;
    float enemySpawnTimer;
    float enemySpawnTimerMax;

    // Game state variables
    int numberOfPlayers;
    int player1Level;
    int player2Level;
    int upperWeaponLevel;
    int lowerWeaponLevel;
    bool upperWeapon;
    bool lowerWeapon;
    float player1Exp;
    float player2Exp;
    float player1ExpNext;
    float player2ExpNext;
    int player1Hp;
    int player2Hp;
    int player1HpMax;
    int player2HpMax;
    int player1Score;
    int player2Score;

    int typeOfBullet;

    bool isReset = false;
    bool countinue = false;
```

```

    // Private member functions
    void loadTextures();
    void createPlayers();
    void updateBullets(Player *p, const float &dt);
    void updateSideGunBullets(Player *p, const float &dt);
    void handleEnemyDeath(Player *p, size_t enemyIndex);

public:
    int times = 0;
    int timesPlayers = 0;
    // Constructor and Destructor
    Game(RenderWindow *window, int numberOfPlayers,
         int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score,
         int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score,
         bool upperWeapon, int upperWeaponLevel, bool lowerWeapon, int lowerWeaponLevel,
         int typeOfBullet);
    virtual ~Game();

    // Accessors
    inline RenderWindow &getWindow() { return *this->window; }

    inline const int &getNumberOfPlayers() const { return this->numberOfPlayers; }
    inline const int &getPlayer1Level() const { return this->player1Level; }
    inline const int &getPlayer2Level() const { return this->player2Level; }
    inline const int &getPlayer1Hp() const { return this->player1Hp; }
    inline const int &getPlayer2Hp() const { return this->player2Hp; }
    inline const int &getPlayer1HpMax() const { return this->player1HpMax; }
    inline const int &getPlayer2HpMax() const { return this->player2HpMax; }
    inline const float &getPlayer1Exp() const { return this->player1Exp; }
    inline const float &getPlayer2Exp() const { return this->player2Exp; }
    inline const float &getPlayer1ExpNext() const { return this->player1ExpNext; }
    inline const float &getPlayer2ExpNext() const { return this->player2ExpNext; }
    inline const int &getPlayer1Score() const { return this->player1Score; }
    inline const int &getPlayer2Score() const { return this->player2Score; }
    inline const bool &getUpperWeapon() const { return this->upperWeapon; }
    inline const bool &getLowerWeapon() const { return this->lowerWeapon; }
    inline const int &getUpperWeaponLevel() const { return this->upperWeaponLevel; }
    inline const int &getLowerWeaponLevel() const { return this->lowerWeaponLevel; }
    inline const int getBulletType() const { return this->players[0]->getCurrentWeapon(); }

    inline const bool &isGameOver() const { return this->gameOver; }
    inline const bool &isGamePaused() const { return this->gamePaused; }
    inline const bool &isInGame() const { return this->inGame; }
    inline bool hasBeenReset() { return this->isReset; }
    inline bool isCountinue() { return this->countinue; }
    inline void pause() { this->gamePaused = true; }
    inline void resume() { this->gamePaused = false; }


    // Modifying functions
    void setIsGameStarted(bool isGameStarted) { this->inGame = isGameStarted; }
    void setWeaponType(int weaponType);
    void setUpperWeapon(bool upperWeapon);
    void setLowerWeapon(bool lowerWeapon);
    void setUpperWeaponLevel(int upperWeaponLevel);
    void setLowerWeaponLevel(int lowerWeaponLevel);

    // Functions
    Player *createPlayer(DynamicArray<Texture> &textures, Vector2u windowBounds,
                         int level, float playerExp, float playerExpNext, int playerHp, int playerHpMax, int playerScore,
                         int upKey, int downKey, int leftKey, int rightKey, int fireKey, int typeOfBullet);

    void getGameState(int &numberOfPlayers,
                      int &player1Level, int &player1Hp, int &player1HpMax, float &player1Exp, float &player1ExpNext, int &player1Score,
                      int &player2Level, int &player2Hp, int &player2HpMax, float &player2Exp, float &player2ExpNext, int &player2Score,
                      bool &upperWeapon, int &upperWeaponLevel,
                      bool &lowerWeapon, int &lowerWeaponLevel);
    void loadSavedGameData(int numberOfPlayers,
                           int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score,
                           int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score,
                           bool upperWeapon, int upperWeaponLevel, bool lowerWeapon, int lowerWeaponLevel, int typeOfBullet);

    void reseted(bool reset) { this->isReset = reset; }
    void restart();
    void exit();

    void initUI();
```

```
    void updateUI(int index);
    void update(const float &dt);
    void renderUI();
    void render();
};
```

6.25 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Menu.hpp File Reference

This is the header file for the Menu class, which is used to create the main menu of the game.

```
#include <SFML/Graphics.hpp>
#include <vector>
#include "./Resources/Button.hpp"
#include "TextTag.hpp"
#include "Game.hpp"
#include <nlohmann/json.hpp>
```

Classes

- class Menu

Typedefs

- using **json** = nlohmann::json

6.25.1 Detailed Description

This is the header file for the Menu class, which is used to create the main menu of the game.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.26 Menu.hpp

Go to the documentation of this file.

```
#pragma once

#include <SFML/Graphics.hpp>
#include <vector>
#include "./Resources/Button.hpp"
#include "TextTag.hpp"
#include "Game.hpp"
#include <nlohmann/json.hpp>

using json = nlohmann::json;

class Menu
{
private:
    Game *game;
    sf::RenderWindow *window;
    sf::Font font;
    sf::Text title;
    sf::Text continueText;
    std::vector<Button> buttons;
    std::vector<Button> beginMenuButtons;
    bool active;
    bool newGame;
    bool beginMenu;

    int numberOfPlayers;
    int player1Level;
    int player2Level;
    int upperWeaponLevel;
    int lowerWeaponLevel;
    bool upperWeapon;
    bool lowerWeapon;
    float player1Exp;
    float player2Exp;
    float player1ExpNext;
    float player2ExpNext;
    int player1Hp;
    int player2Hp;
    int player1HpMax;
    int player2HpMax;
    int player1Score;
    int player2Score;

    int typeOfBullet;

public:
    Menu(sf::RenderWindow *window, Game *game);
    void render(sf::RenderTarget &target);
    void update(const sf::Vector2f &mousePos, sf::Event &event);
    void addButton(float x, float y, float width, float height, float cornerRadius, sf::Font &font, const std::string &text);
    void addButtonToBeginMenu(float x, float y, float width, float height, float cornerRadius, sf::Font &font, const std::string &text);
    void resetColor();
    void resetUpperLevelColor();
    void resetLowerLevelColor();
    void resetAllColor();
    void setNewGame(bool newGame)
    {
        this->newGame = newGame;
        this->beginMenu = false;
    };

    bool isButtonPressed(int index) const;
};
```

6.27 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Player.hpp File Reference

This is the header file for the Player class, which is used to create the player object.

```
#include <vector>
#include <iostream>
```

```
#include "./Bullets/Bullet.hpp"
#include "../include/Weapons/Weapon.hpp"
#include "../include/Weapons/PeaShooter.hpp"
#include "./Resources/Fonts.hpp"
#include "./Resources/Textures.hpp"
#include "../include/Bullets/LaserBullet.hpp"
#include "../include/Bullets/Lightning.hpp"
#include "../include/Bullets/DarkMatter.hpp"
#include "../include/Bullets/NuclierMaterial.hpp"
#include "../include/Bullets/Plasma.hpp"
#include "../include/Bullets/PlanetaryBomb.hpp"
```

Classes

- class Player

Enumerations

- enum **controls** {
  UP = 0 , **DOWN** , **LEFT** , **RIGHT** ,
  FIRE }
- enum **mainWeapons** {
  LASER = 0 , **LIGHTNING** , **DARK_MATTER** , **NUCLIER_MATERIAL** ,
  PLASMA , **PLANETARY_BOMB** }
- enum **sideWeapons** { **WEAPON_UP** = 0 , **WEAPON_DOWN** }

6.27.1 Detailed Description

This is the header file for the Player class, which is used to create the player object.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.28 Player.hpp

Go to the documentation of this file.

```
#pragma once
#include <vector>
#include <iostream>

#include "./Bullets/Bullet.hpp"
#include "../include/Weapons/Weapon.hpp"
#include "../include/Weapons/PeaShooter.hpp"
#include "./Resources/Fonts.hpp"
#include "./Resources/Textures.hpp"

#include "../include/Bullets/LaserBullet.hpp"
#include "../include/Bullets/Lightning.hpp"
#include "../include/Bullets/DarkMatter.hpp"
#include "../include/Bullets/NuclierMaterial.hpp"
#include "../include/Bullets/Plasma.hpp"
#include "../include/Bullets/PlanetaryBomb.hpp"

class Player
{
private:
    unsigned playerNumber;
    float dtMultiplier;

    // Player sprite
    Sprite sprite;
    RectangleShape hitbox;

    Vector2u windowBounds;

    // Bullets
    DynamicArray<Bullet *> bullets;

    // Controls
    int controls[5];
    Vector2f direction;
    Vector2f currentVelocity;
    float maxVelocity;
    float acceleration;
    float drag;

    // Player stats
    float fireRate;
    float fireRateMax;

    float damageTimer;
    float damageTimerMax;
    int damage;
    int damageMax;

    int level;
    int maxLevel;
    bool isAtMaxLevel = false;

    float exp;
    float expNext;
    int statPoints;

    int hp;
    int hpMax;

    int score;

    int endurance;
    int armor;
    int strength;
    int agility;
    int upgrade;

    // Weapons
    DynamicArray<Weapon *> weapons;
    Weapon *sideGunUp;
    Weapon *sideGunDown;

    int currentWeapon;

    bool addedUpperWeapon = false;
    bool addedLowerWeapon = false;

    // Upgarde system
    int sideGunLevel = 0;

```

```
    bool uperWeapon;
    bool lowerWeapon;
    int upperWeaponLevel;
    int lowerWeaponLevel;

public:
    // Constructor
    Player(DynamicArray<Texture> &textures, Vector2u windowBounds, int typeOfBullet,
           int UP = 22, int DOWN = 18, int LEFT = 0, int RIGHT = 3, int FIRE = 57,
           int playerLevel = 0, float playerExp = 0, float playerExpNext = 16, int playerHp = 10, int playerHpMax = 10, int playerScore = 0,
           int upperWeaponLevel = 0, int lowerWeaponLevel = 0,
           bool uperWeapon = false, bool lowerWeapon = false);

    ~Player();

    // Accessors
    Bullet &getBullet(unsigned index);
    void removeBullet(unsigned index);
    inline const int getBulletSize() { return this->bullets.size(); }
    inline const Vector2f &getPosition() const { return this->sprite.getPosition(); }
    inline DynamicArray<Weapon *> &getWeapons() { return this->weapons; };
    inline const String getHpS() const { return std::to_string(this->hp) + "/" + std::to_string(this->hpMax); }
    inline const FloatRect getBounds() const { return this->sprite.getGlobalBounds(); }
    inline const bool isDead() const { return this->hp <= 0; }
    inline const int getPlayerNumber() const { return this->playerNumber; }
    inline const int getLevel() const { return this->level; }
    inline const int getMaxLevel() const { return this->maxLevel; }
    inline const bool getIsAtMaxLevel() const { return this->isAtMaxLevel; }
    inline const int getExp() const { return this->exp; }
    inline const int getExpNext() const { return this->expNext; }
    inline const int getStatPoints() const { return this->statPoints; }
    inline const int getHp() const { return this->hp; }
    inline const int getHpMax() const { return this->hpMax; }
    inline const int getScore() const { return this->score; }
    inline const int getCurrentWeapon() const { return this->currentWeapon; }
    inline bool gainExp(float exp)
    {
        if (!this->isAtMaxLevel)
        {
            this->exp += exp;
            return this->updateLevelingSystem();
        }
        return false;
    }

    // Templait function adding to bullet vector
    template <typename T>
    void setBulletType(Vector2f pos, int upgrade, int level, Vector2f directionUp = Vector2f(1.f, -0.1f), Vector2f directionDown = Vector2f(1.f, 0.1f),
                       float initialVelocity = 2.f, float maxVelocity = 50.f, float acceleration = 1.f);

    // Modifiers
    void setToDefault();
    void setCurrentWeapon(int weapon) { this->currentWeapon = weapon; }
    void setUpperWeaponLevel(int level) { this->upperWeaponLevel = level; }
    void setLowerWeaponLevel(int level) { this->lowerWeaponLevel = level; }
    void setUperWeapon(bool uperWeapon) { this->uperWeapon = uperWeapon; }
    void setLowerWeapon(bool lowerWeapon) { this->lowerWeapon = lowerWeapon; }

    void setLevel(int level) { this->level = level; }
    void setExp(float exp) { this->exp = exp; }
    void setExpNext(float expNext) { this->expNext = expNext; }
    void setStatPoints(int statPoints) { this->statPoints = statPoints; }
    void setHp(int hp) { this->hp = hp; }
    void setHpMax(int hpMax) { this->hpMax = hpMax; }
    void setScore(int score) { this->score = score; }

    // Functionsc
    void Move(const float &dt);
    bool updateLevelingSystem();
    void LevelUp();
    void addWeapon(int type, int UpOrDown, int level);
    void CombatUpdate();
    void UpdateAccessories(const float &dt);
    void TakeDamage(int damage);
    void Update(Vector2u windowBounds, const float &dt);
    void Render(RenderTarget &target);

    static unsigned players;
};

// Enumerations
enum controls
{
```

```
    UP = 0,
    DOWN,
    LEFT,
    RIGHT,
    FIRE
};
enum mainWeapons
{
    LASER = 0,
    LIGHTNING,
    DARK_MATTER,
    NUCLIER_MATERIAL,
    PLASMA,
    PLANETARY_BOMB
};

enum sideWeapons
{
    WEAPON_UP = 0,
    WEAPON_DOWN
};
```

6.29 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Resources/BulletResources.hpp File Reference

This is the header file for the BulletResources enum, which is used to store the paths to the bullet sprites and their damage values.

```
#include <unordered_map>
#include <string>
```

Enumerations

- enum **BulletLevels** {
  LEVEL1 = 0 , **LEVEL2** , **LEVEL3** , **LEVEL4** ,
  LEVEL5 , **NUM_BULLET_LEVELS** }
- enum **ChangingBullets** {
  LASER_BULLET = 0 , **LIGHTNING_BULLET** , **DARK_MATTER_BULLET** , **NUCLIER_MATERIAL_↩ BULLET** ,
  PLASMA_BULLET , **PLANETARY_BOMB_BULLET** , **PEA_SHOOTER_BULLET** , **NUM_CHANGING_↩ BULLETS** }

Variables

- const std::unordered_map< BulletLevels, std::string > resourcePathsLaserBullets
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsBulletsDamage
- const std::unordered_map< BulletLevels, std::string > resourcePathsLightningBullets
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsLightningBulletsDamage
- const std::unordered_map< BulletLevels, std::string > resourcePathsDarkMatterBullet
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsDarkMatterBulletDamage
- const std::unordered_map< BulletLevels, std::string > resourcePathsNuclierMaterialBullet
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsNuclierMaterialBulletDamage
- const std::unordered_map< BulletLevels, std::string > resourcePathsPlasmaBullets
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsPlasmaBulletsDamage
- const std::unordered_map< BulletLevels, std::string > resourcePathsPlanetaryBombBullets
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsPlanetaryBombBulletsDamage

- const std::unordered_map< BulletLevels, std::string > resourcePathsPeaShooterBullets
- const std::unordered_map< BulletLevels, unsigned int > resourcePathsPeaShooterBulletsDamage
- const std::unordered_map< ChangingBullets, std::unordered_map< BulletLevels, std::string > > resourcePathsMaps
- const std::unordered_map< ChangingBullets, std::unordered_map< BulletLevels, unsigned int > > resourcePathsDamageMaps

6.29.1 Detailed Description

This is the header file for the BulletResources enum, which is used to store the paths to the bullet sprites and their damage values.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.29.2 Variable Documentation

6.29.2.1 resourcePathsBulletsDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsBulletsDamage
```

Initial value:

```
= {
    {LEVEL1, 1},
    {LEVEL2, 1},
    {LEVEL3, 1},
    {LEVEL4, 1},
    {LEVEL5, 1},
}
```

6.29.2.2 resourcePathsDamageMaps

```
const std::unordered_map<ChangingBullets, std::unordered_map<BulletLevels, unsigned int> >
resourcePathsDamageMaps
```

Initial value:

```
= {

    {LASER_BULLET, resourcePathsBulletsDamage},
    {LIGHTNING_BULLET, resourcePathsLightningBulletsDamage},
    {DARK_MATTER_BULLET, resourcePathsDarkMatterBulletDamage},
    {NUCLIER_MATERIAL_BULLET, resourcePathsNuclierMaterialBulletDamage},
    {PLASMA_BULLET, resourcePathsPlasmaBulletsDamage},
    {PLANETARY_BOMB_BULLET, resourcePathsPlanetaryBombBulletsDamage},
    {PEA_SHOOTER_BULLET, resourcePathsPeaShooterBulletsDamage},
}
```

6.29.2.3 resourcePathsDarkMatterBullet

```
const std::unordered_map<BulletLevels, std::string> resourcePathsDarkMatterBullet
```

Initial value:

```
= {

    {LEVEL1, "include/Resources/Sprites/LaserBullets/DarkMatterLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/DarkMatterLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/DarkMatterLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/DarkMatterLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/DarkMatterLevel5.png"},
}
```

6.29.2.4 resourcePathsDarkMatterBulletDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsDarkMatterBulletDamage
```

Initial value:

```
= {
    {LEVEL1, 2},
    {LEVEL2, 3},
    {LEVEL3, 4},
    {LEVEL4, 5},
    {LEVEL5, 6}}
```

6.29.2.5 resourcePathsLaserBullets

```
const std::unordered_map<BulletLevels, std::string> resourcePathsLaserBullets
```

Initial value:

```
= {
    {LEVEL1, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/Bullet.png"},
}
```

6.29.2.6 resourcePathsLightningBullets

```
const std::unordered_map<BulletLevels, std::string> resourcePathsLightningBullets
```

Initial value:

```
= {

    {LEVEL1, "include/Resources/Sprites/LaserBullets/LightningLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/LightningLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/LightningLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/LightningLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/LightningLevel5.png"},
}
```

6.29.2.7 resourcePathsLightningBulletsDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsLightningBulletsDamage
```

Initial value:

```
= {
    {LEVEL1, 1},
    {LEVEL2, 2},
    {LEVEL3, 3},
    {LEVEL4, 4},
    {LEVEL5, 5}}
```

6.29.2.8 resourcePathsMaps

```
const std::unordered_map<ChangingBullets, std::unordered_map<BulletLevels, std::string> >
resourcePathsMaps
```

Initial value:

```
= {

    {LASER_BULLET, resourcePathsLaserBullets},
    {LIGHTNING_BULLET, resourcePathsLightningBullets},
    {DARK_MATTER_BULLET, resourcePathsDarkMatterBullet},
    {NUCLIER_MATERIAL_BULLET, resourcePathsNuclierMaterialBullet},
    {PLASMA_BULLET, resourcePathsPlasmaBullets},
    {PLANETARY_BOMB_BULLET, resourcePathsPlanetaryBombBullets},
    {PEA_SHOOTER_BULLET, resourcePathsPeaShooterBullets},
}
```

6.29.2.9 resourcePathsNuclierMaterialBullet

```
const std::unordered_map<BulletLevels, std::string> resourcePathsNuclierMaterialBullet
```

Initial value:

```
= {

    {LEVEL1, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel5.png"},
}
```

6.29.2.10 resourcePathsNuclierMaterialBulletDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsNuclierMaterialBulletDamage
```

Initial value:

```
= {
    {LEVEL1, 3},
    {LEVEL2, 4},
    {LEVEL3, 5},
    {LEVEL4, 6},
    {LEVEL5, 7}}
```

6.29.2.11 resourcePathsPeaShooterBullets

```
const std::unordered_map<BulletLevels, std::string> resourcePathsPeaShooterBullets
```

Initial value:

```
= {
    {LEVEL1, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/01.png"},
}
```

6.29.2.12 resourcePathsPeaShooterBulletsDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsPeaShooterBulletsDamage
```

Initial value:

```
= {
    {LEVEL1, 1},
    {LEVEL2, 1},
    {LEVEL3, 1},
    {LEVEL4, 1},
    {LEVEL5, 1},
}
```

6.29.2.13 resourcePathsPlanetaryBombBullets

```
const std::unordered_map<BulletLevels, std::string> resourcePathsPlanetaryBombBullets
```

Initial value:

```
= {

    {LEVEL1, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel5.png"},
}
```

6.29.2.14 resourcePathsPlanetaryBombBulletsDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsPlanetaryBombBulletsDamage
```

Initial value:

```
= {
    {LEVEL1, 5},
    {LEVEL2, 6},
    {LEVEL3, 7},
    {LEVEL4, 8},
    {LEVEL5, 9}}
```

6.29.2.15 resourcePathsPlasmaBullets

```
const std::unordered_map<BulletLevels, std::string> resourcePathsPlasmaBullets
```

Initial value:

```
= {

    {LEVEL1, "include/Resources/Sprites/LaserBullets/PlasmaLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/PlasmaLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/PlasmaLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/PlasmaLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/PlasmaLevel5.png"},
}
```

6.29.2.16 resourcePathsPlasmaBulletsDamage

```
const std::unordered_map<BulletLevels, unsigned int> resourcePathsPlasmaBulletsDamage
```

Initial value:

```
= {
    {LEVEL1, 4},
    {LEVEL2, 5},
    {LEVEL3, 6},
    {LEVEL4, 7},
    {LEVEL5, 8}}
```

6.30 BulletResources.hpp

Go to the documentation of this file.

```
#pragma once

#include <unordered_map>
#include <string>

enum BulletLevels
{
    // Bullets
    LEVEL1 = 0,

    LEVEL2,
    LEVEL3,
    LEVEL4,
    LEVEL5,

    // Total
    NUM_BULLET_LEVELS
};

const std::unordered_map<BulletLevels, std::string> resourcePathsLaserBullets = {
    {LEVEL1, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/Bullet.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/Bullet.png"},
};
const std::unordered_map<BulletLevels, unsigned int> resourcePathsBulletsDamage = {
    {LEVEL1, 1},
    {LEVEL2, 1},
    {LEVEL3, 1},
    {LEVEL4, 1},
    {LEVEL5, 1},
};


const std::unordered_map<BulletLevels, std::string> resourcePathsLightningBullets = {
    // Player
    {LEVEL1, "include/Resources/Sprites/LaserBullets/LightningLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/LightningLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/LightningLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/LightningLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/LightningLevel5.png"},
};

// Enum for all bullet damage types

const std::unordered_map<BulletLevels, unsigned int> resourcePathsLightningBulletsDamage = {
    {LEVEL1, 1},
    {LEVEL2, 2},
    {LEVEL3, 3},
    {LEVEL4, 4},
    {LEVEL5, 5}};

const std::unordered_map<BulletLevels, std::string> resourcePathsDarkMatterBullet = {
    // Player
    {LEVEL1, "include/Resources/Sprites/LaserBullets/DarkMatterLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/DarkMatterLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/DarkMatterLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/DarkMatterLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/DarkMatterLevel5.png"},
};

const std::unordered_map<BulletLevels, unsigned int> resourcePathsDarkMatterBulletDamage = {
    {LEVEL1, 2},
    {LEVEL2, 3},
    {LEVEL3, 4},
    {LEVEL4, 5},
    {LEVEL5, 6}};

const std::unordered_map<BulletLevels, std::string> resourcePathsNuclierMaterialBullet = {
    // Player
    {LEVEL1, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/NuclierMaterialLevel5.png"},
};

const std::unordered_map<BulletLevels, unsigned int> resourcePathsNuclierMaterialBulletDamage = {
    {LEVEL1, 3},
    {LEVEL2, 4},
```

```
    {LEVEL3, 5},
    {LEVEL4, 6},
    {LEVEL5, 7}};

const std::unordered_map<BulletLevels, std::string> resourcePathsPlasmaBullets = {
    // Player
    {LEVEL1, "include/Resources/Sprites/LaserBullets/PlasmaLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/PlasmaLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/PlasmaLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/PlasmaLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/PlasmaLevel5.png"},
};

const std::unordered_map<BulletLevels, unsigned int> resourcePathsPlasmaBulletsDamage = {
    {LEVEL1, 4},
    {LEVEL2, 5},
    {LEVEL3, 6},
    {LEVEL4, 7},
    {LEVEL5, 8}};

const std::unordered_map<BulletLevels, std::string> resourcePathsPlanetaryBombBullets = {
    // Player
    {LEVEL1, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel1.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel2.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel3.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel4.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/PlanetaryBombLevel5.png"},
};

const std::unordered_map<BulletLevels, unsigned int> resourcePathsPlanetaryBombBulletsDamage = {
    {LEVEL1, 5},
    {LEVEL2, 6},
    {LEVEL3, 7},
    {LEVEL4, 8},
    {LEVEL5, 9}};

const std::unordered_map<BulletLevels, std::string> resourcePathsPeaShooterBullets = {
    {LEVEL1, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL2, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL3, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL4, "include/Resources/Sprites/LaserBullets/01.png"},
    {LEVEL5, "include/Resources/Sprites/LaserBullets/01.png"},
};
const std::unordered_map<BulletLevels, unsigned int> resourcePathsPeaShooterBulletsDamage = {
    {LEVEL1, 1},
    {LEVEL2, 1},
    {LEVEL3, 1},
    {LEVEL4, 1},
    {LEVEL5, 1},
};

enum ChangingBullets
{
    // Bullets
    LASER_BULLET = 0,
    LIGHTNING_BULLET,
    DARK_MATTER_BULLET,
    NUCLIER_MATERIAL_BULLET,
    PLASMA_BULLET,
    PLANETARY_BOMB_BULLET,

    // Side Gun
    PEA_SHOOTER_BULLET,

    // Total
    NUM_CHANGING_BULLETS
};

const std::unordered_map<ChangingBullets, std::unordered_map<BulletLevels, std::string> >
resourcePathsMaps = {
    // Player
    {LASER_BULLET, resourcePathsLaserBullets},
    {LIGHTNING_BULLET, resourcePathsLightningBullets},
    {DARK_MATTER_BULLET, resourcePathsDarkMatterBullet},
    {NUCLIER_MATERIAL_BULLET, resourcePathsNuclierMaterialBullet},
    {PLASMA_BULLET, resourcePathsPlasmaBullets},
    {PLANETARY_BOMB_BULLET, resourcePathsPlanetaryBombBullets},
    {PEA_SHOOTER_BULLET, resourcePathsPeaShooterBullets},
};

const std::unordered_map<ChangingBullets, std::unordered_map<BulletLevels, unsigned int> >
resourcePathsDamageMaps = {
    // Player
    {LASER_BULLET, resourcePathsBulletsDamage},
    {LIGHTNING_BULLET, resourcePathsLightningBulletsDamage},
    {DARK_MATTER_BULLET, resourcePathsDarkMatterBulletDamage},
    {NUCLIER_MATERIAL_BULLET, resourcePathsNuclierMaterialBulletDamage},
```

```
    {PLASMA_BULLET, resourcePathsPlasmaBulletsDamage},
    {PLANETARY_BOMB_BULLET, resourcePathsPlanetaryBombBulletsDamage},
    {PEA_SHOOTER_BULLET, resourcePathsPeaShooterBulletsDamage},
};
```

6.31 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Resources/Button.hpp File Reference

This is the header file for the Button class, which is used to create buttons for the game.

```
#include <SFML/Graphics.hpp>
#include <functional>
#include <string>
#include <iostream>
```

Classes

- class Button

  A class to represent a button in the game.

6.31.1 Detailed Description

This is the header file for the Button class, which is used to create buttons for the game.

Author

Klaoyan

Version

0.1

Date

2024-05-30

The Button class provides functionalities to create interactive buttons within the game, including setting their position, size, colors for different states (idle, hover, active), and handling click events.

Note

This file requires the SFML library.

Copyright

Copyright (c) 2024

6.32 Button.hpp

Go to the documentation of this file.

```
#pragma once

#include <SFML/Graphics.hpp>
#include <functional>
#include <string>
#include <iostream>

class Button
{
public:
    Button(float x, float y, float width, float height, float radius, sf::Font &font, const std::string &text,
            const sf::Color &idleColor, const sf::Color &hoverColor, const sf::Color &activeColor);

    ~Button();

    void setOnClick(std::function<void()> func);

    inline bool isClicked(const sf::Vector2f &mousePos) const { return shape.getGlobalBounds().contains(mousePos); };

    inline const std::string getText() const { return text.getString(); }

    void update(const sf::Vector2f &mousePos);

    void render(sf::RenderTarget &target);

    void changeColor(const sf::Color &idleColor, const sf::Color &hoverColor, const sf::Color &activeColor)
    {
        this->idleColor = idleColor;
        this->hoverColor = hoverColor;
        this->activeColor = activeColor;
    }

    int timesPressed = 0;

private:
    sf::ConvexShape shape;
    sf::Font &font;
    sf::Text text;
    sf::Color idleColor;
    sf::Color hoverColor;
    sf::Color activeColor;
    sf::Color changeColorTo;
    std::function<void()> onClick;

    enum ButtonState
    {
        BTN_IDLE,
        BTN_HOVER,
        BTN_ACTIVE
    };
    ButtonState buttonState;

    bool wasPressed = false;

    void updateColor();

    void setRoundedRectangle(sf::ConvexShape &shape, float width, float height, float radius);
};
```

6.33 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Resources/DynamicArray.hpp File Reference

This is the header file for the DynamicArray class, which is used to create a dynamic array of elements of any type.

```
#include <stdexcept>
#include <vector>
```

Classes

- class DynamicArray< T >

6.33.1 Detailed Description

This is the header file for the DynamicArray class, which is used to create a dynamic array of elements of any type.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.34 DynamicArray.hpp

Go to the documentation of this file.

```
00001
00011 #include <stdexcept>
00012 #include <vector>
00013
00014 template <typename T>
00015 class DynamicArray
00016 {
00017 private:
00018     unsigned capacity;
00019     unsigned current_size;
00020     T **elements;
00021
00022     void initialize(unsigned from);
00023     void expand();
00024
00025 public:
00026     DynamicArray(unsigned initialCapacity = 5);
00027     DynamicArray(const DynamicArray &other);
00028     ~DynamicArray();
00029
00030     T &operator[](int index);
00031     void operator=(const DynamicArray &other);
00032
00033     void add(const T &element);
00034     void remove(int index);
00035     unsigned size() const;
00036     void clear();
00037
00038     // Iterator typedefs
00039     typedef T **iterator;
00040     typedef const T **const_iterator;
00041
00042     // Iterator functions
00043     iterator begin() { return elements; }
00044     iterator end() { return elements + current_size; }
00045     const_iterator begin() const { return elements; }
00046     const_iterator end() const { return elements + current_size; }
```

```
};

template <typename T>
DynamicArray<T>::DynamicArray(unsigned initialCapacity)
    : capacity(initialCapacity), current_size(0)
{
    elements = new T *[capacity];
    initialize(0);
}

template <typename T>
DynamicArray<T>::DynamicArray(const DynamicArray &other)
    : capacity(other.capacity), current_size(other.current_size)
{
    elements = new T *[capacity];
    for (unsigned i = 0; i < current_size; ++i)
    {
        elements[i] = new T(*other.elements[i]);
    }
    initialize(current_size);
}

template <typename T>
DynamicArray<T>::~DynamicArray()
{
    clear();
    delete[] elements;
}

template <typename T>
void DynamicArray<T>::initialize(unsigned from)
{
    for (unsigned i = from; i < capacity; ++i)
    {
        elements[i] = nullptr;
    }
}

template <typename T>
void DynamicArray<T>::expand()
{
    capacity *= 2;
    T **temp = new T *[capacity];
    for (unsigned i = 0; i < current_size; ++i)
    {
        temp[i] = elements[i];
    }
    delete[] elements;
    elements = temp;
    initialize(current_size);
}

template <typename T>
T &DynamicArray<T>::operator[](int index)
{
    if (index < 0 || static_cast<unsigned>(index) >= current_size)
    {
        throw std::out_of_range("Index out of bounds");
    }
    return *elements[index];
}

template <typename T>
void DynamicArray<T>::operator=(const DynamicArray &other)
{
    if (this == &other)
    {
        return;
    }
    clear();
    delete[] elements;
    capacity = other.capacity;
    current_size = other.current_size;
    elements = new T *[capacity];
    for (unsigned i = 0; i < current_size; ++i)
    {
        elements[i] = new T(*other.elements[i]);
    }
    initialize(current_size);
}

template <typename T>
void DynamicArray<T>::add(const T &element)
{
    if (current_size >= capacity)
    {
        expand();
```

```
    }
    elements[current_size++] = new T(element);
}

template <typename T>
void DynamicArray<T>::remove(int index)
{
    if (index < 0 || static_cast<unsigned>(index) >= current_size)
    {
        throw std::out_of_range("Index out of bounds");
    }
    delete elements[index];

    if (index != current_size - 1)
    {
        elements[index] = elements[current_size - 1];
    }
    elements[--current_size] = nullptr;
}

template <typename T>
unsigned DynamicArray<T>::size() const
{
    return current_size;
}

template <typename T>
void DynamicArray<T>::clear()
{
    for (unsigned i = 0; i < current_size; ++i)
    {
        delete elements[i];
        elements[i] = nullptr;
    }
    current_size = 0;
}
```

6.35 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Resources/Fonts.hpp File Reference

This is the header file for the Fonts enum, which is used to store the paths to the fonts used in the game.

```
#include <unordered_map>
#include <string>
```

Enumerations

- enum **Fonts** { **DOSIS_LIGHT** = 0 , **GAME_OVER** , **FONTS_COUNT** }

Variables

- const std::unordered_map< Fonts, std::string > resourcePathsFonts

6.35.1 Detailed Description

This is the header file for the Fonts enum, which is used to store the paths to the fonts used in the game.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.35.2 Variable Documentation

6.35.2.1 resourcePathsFonts

```
const std::unordered_map<Fonts, std::string> resourcePathsFonts
```

Initial value:

```
= {

    {DOSIS_LIGHT, "Fonts/Dosis-Light.ttf"},
    {GAME_OVER, "Fonts/GAME_glm.ttf"},


}
```

6.36 Fonts.hpp

Go to the documentation of this file.

```
#pragma once

#include <unordered_map>
#include <string>

enum Fonts
{
    DOSIS_LIGHT = 0,
    GAME_OVER,
    FONTS_COUNT
};
const std::unordered_map<Fonts, std::string> resourcePathsFonts = {

    {DOSIS_LIGHT, "Fonts/Dosis-Light.ttf"},
    {GAME_OVER, "Fonts/GAME_glm.ttf"},


};
```

6.37 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Resources/SaveData.hpp File Reference

This is the header file for the SaveData class, which is used to save and load data from the game.

```
#include <nlohmann/json.hpp>
```

Typedefs

- using **json** = nlohmann::json

Functions

- void saveData (int numberOfPlayers, int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score, int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score, bool upperWeapon, int upperWeaponLevel, bool lowerWeapon, int lower↩ WeaponLevel, int typeOfBullet)

  Saves game data to a JSON file.
- void loadSavedData (int &numberOfPlayers, int &player1Level, int &player1Hp, int &player1HpMax, float &player1Exp, float &player1ExpNext, int &player1Score, int &player2Level, int &player2Hp, int &player2↩ HpMax, float &player2Exp, float &player2ExpNext, int &player2Score, bool &upperWeapon, int &upper↩ WeaponLevel, bool &lowerWeapon, int &lowerWeaponLevel, int &typeOfBullet)

  Loads game data from a JSON file.
- void saveDefaultData ()

  Saves default game data to a JSON file.
- void eraceSavedData ()

  Erases saved game data by replacing it with default data.
- void setNumberOfPlayers (int numberOfPlayers)

  Sets the number of players in the saved game data.
- void setPlayer1Level (int player1Level)

  Sets the level of player 1 in the saved game data.
- void setPlayer1Hp (int player1Hp)

  Sets the HP of player 1 in the saved game data.
- void setPlayer1HpMax (int player1HpMax)

  Sets the maximum HP of player 1 in the saved game data.
- void setPlayer1Exp (float player1Exp)

  Sets the experience points of player 1 in the saved game data.
- void setPlayer1ExpNext (float player1ExpNext)

  Sets the next level experience points of player 1 in the saved game data.
- void setPlayer1Score (int player1Score)

  Sets the score of player 1 in the saved game data.
- void setPlayer2Level (int player2Level)

  Sets the level of player 2 in the saved game data.
- void setPlayer2Hp (int player2Hp)

  Sets the HP of player 2 in the saved game data.
- void setPlayer2HpMax (int player2HpMax)

  Sets the maximum HP of player 2 in the saved game data.
- void setPlayer2Exp (float player2Exp)

  Sets the experience points of player 2 in the saved game data.
- void setPlayer2ExpNext (float player2ExpNext)

  Sets the next level experience points of player 2 in the saved game data.
- void setPlayer2Score (int player2Score)

  Sets the score of player 2 in the saved game data.
- void setUpperWeapon (bool upperWeapon)

  Sets whether the upper weapon is enabled in the saved game data.
- void setUpperWeaponLevel (int upperWeaponLevel)

  Sets the level of the upper weapon in the saved game data.
- void setLowerWeapon (bool lowerWeapon)

Sets whether the lower weapon is enabled in the saved game data.

- void setLowerWeaponLevel (int lowerWeaponLevel)

  Sets the level of the lower weapon in the saved game data.
- void setTypeOfBullet (int typeOfBullet)

  Sets the type of bullet in the saved game data.
- void setContinueGame (bool continueGame)

  Sets whether the game continues from the saved state in the saved game data.

6.37.1 Detailed Description

This is the header file for the SaveData class, which is used to save and load data from the game.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.37.2 Function Documentation

6.37.2.1 eraceSavedData()

```
void eraceSavedData ()
```

Erases saved game data by replacing it with default data.

Create default data

Open file to save default data

Write default data to file

6.37.2.2 loadSavedData()

```
void loadSavedData (
            int & numberOfPlayers,
            int & player1Level,
            int & player1Hp,
            int & player1HpMax,
            float & player1Exp,
            float & player1ExpNext,
            int & player1Score,
            int & player2Level,
            int & player2Hp,
            int & player2HpMax,
            float & player2Exp,
            float & player2ExpNext,
            int & player2Score,
            bool & upperWeapon,
            int & upperWeaponLevel,
            bool & lowerWeapon,
            int & lowerWeaponLevel,
            int & typeOfBullet)
```

Loads game data from a JSON file.

Parameters

numberOfPlayers	The number of players.
player1Level	Player 1's level.
player1Hp	Player 1's health points.
player1HpMax	Player 1's maximum health points.
player1Exp	Player 1's experience.
player1ExpNext	Player 1's experience required for next level.
player1Score	Player 1's score.
player2Level	Player 2's level.
player2Hp	Player 2's health points.
player2HpMax	Player 2's maximum health points.
player2Exp	Player 2's experience.
player2ExpNext	Player 2's experience required for next level.
player2Score	Player 2's score.
upperWeapon	Whether the upper weapon is enabled.
upperWeaponLevel	The level of the upper weapon.
lowerWeapon	Whether the lower weapon is enabled.
lowerWeaponLevel	The level of the lower weapon.
typeOfBullet	The type of bullet.

Open file to load data

Read data from file

Extract game data

6.37.2.3 saveData()

```
void saveData (
            int numberOfPlayers,
            int player1Level,
            int player1Hp,
            int player1HpMax,
            float player1Exp,
            float player1ExpNext,
            int player1Score,
            int player2Level,
            int player2Hp,
            int player2HpMax,
            float player2Exp,
            float player2ExpNext,
            int player2Score,
            bool upperWeapon,
            int upperWeaponLevel,
            bool lowerWeapon,
            int lowerWeaponLevel,
            int typeOfBullet)
```

Saves game data to a JSON file.

Parameters

numberOfPlayers	The number of players.
player1Level	Player 1's level.
player1Hp	Player 1's health points.
player1HpMax	Player 1's maximum health points.
player1Exp	Player 1's experience.
player1ExpNext	Player 1's experience required for next level.
player1Score	Player 1's score.
player2Level	Player 2's level.
player2Hp	Player 2's health points.
player2HpMax	Player 2's maximum health points.
player2Exp	Player 2's experience.
player2ExpNext	Player 2's experience required for next level.
player2Score	Player 2's score.
upperWeapon	Whether the upper weapon is enabled.
upperWeaponLevel	The level of the upper weapon.
lowerWeapon	Whether the lower weapon is enabled.
lowerWeaponLevel	The level of the lower weapon.
typeOfBullet	The type of bullet.

6.37.2.4 saveDefaultData()

```
void saveDefaultData ()
```

Saves default game data to a JSON file.

Open file to load data

Read data from file

Update default data

Save default data

6.37.2.5 setContinueGame()

```
void setContinueGame (
            bool continueGame)
```

Sets whether the game continues from the saved state in the saved game data.

Parameters

continueGame	Whether the game continues.

6.37.2.6 setLowerWeapon()

```
void setLowerWeapon (
            bool lowerWeapon)
```

Sets whether the lower weapon is enabled in the saved game data.

Parameters

lowerWeapon	Whether the lower weapon is enabled.

6.37.2.7 setLowerWeaponLevel()

```
void setLowerWeaponLevel (
            int lowerWeaponLevel)
```

Sets the level of the lower weapon in the saved game data.

Parameters

lowerWeaponLevel	The level of the lower weapon.

6.37.2.8 setNumberOfPlayers()

```
void setNumberOfPlayers (
            int numberOfPlayers)
```

Sets the number of players in the saved game data.

Parameters

numberOfPlayers	The number of players.

6.37.2.9 setPlayer1Exp()

```
void setPlayer1Exp (
            float player1Exp)
```

Sets the experience points of player 1 in the saved game data.

Parameters

player1Exp	The experience points of player 1.

6.37.2.10 setPlayer1ExpNext()

```
void setPlayer1ExpNext (
            float player1ExpNext)
```

Sets the next level experience points of player 1 in the saved game data.

Parameters

player1ExpNext	The next level experience points of player 1.

6.37.2.11 setPlayer1Hp()

```
void setPlayer1Hp (
            int player1Hp)
```

Sets the HP of player 1 in the saved game data.

Parameters

player1Hp	The HP of player 1.

6.37.2.12 setPlayer1HpMax()

```
void setPlayer1HpMax (
            int player1HpMax)
```

Sets the maximum HP of player 1 in the saved game data.

Parameters

player1HpMax	The maximum HP of player 1.

6.37.2.13 setPlayer1Level()

```
void setPlayer1Level (
            int player1Level)
```

Sets the level of player 1 in the saved game data.

Parameters

player1Level	The level of player 1.

6.37.2.14 setPlayer1Score()

```
void setPlayer1Score (
            int player1Score)
```

Sets the score of player 1 in the saved game data.

Parameters

player1Score	The score of player 1.

6.37.2.15 setPlayer2Exp()

```
void setPlayer2Exp (
            float player2Exp)
```

Sets the experience points of player 2 in the saved game data.

Parameters

player2Exp	The experience points of player 2.

6.37.2.16 setPlayer2ExpNext()

```
void setPlayer2ExpNext (
            float player2ExpNext)
```

Sets the next level experience points of player 2 in the saved game data.

Parameters

player2ExpNext	The next level experience points of player 2.

6.37.2.17 setPlayer2Hp()

```
void setPlayer2Hp (
            int player2Hp)
```

Sets the HP of player 2 in the saved game data.

Parameters

player2Hp	The HP of player 2.

6.37.2.18 setPlayer2HpMax()

```
void setPlayer2HpMax (
            int player2HpMax)
```

Sets the maximum HP of player 2 in the saved game data.

Parameters

player2HpMax	The maximum HP of player 2.

6.37.2.19 setPlayer2Level()

```
void setPlayer2Level (
            int player2Level)
```

Sets the level of player 2 in the saved game data.

Parameters

player2Level	The level of player 2.

6.37.2.20 setPlayer2Score()

```
void setPlayer2Score (
            int player2Score)
```

Sets the score of player 2 in the saved game data.

Parameters

player2Score	The score of player 2.

6.37.2.21 setTypeOfBullet()

```
void setTypeOfBullet (
            int typeOfBullet)
```

Sets the type of bullet in the saved game data.

Parameters

typeOfBullet	The type of bullet.

6.37.2.22 setUpperWeapon()

```
void setUpperWeapon (
            bool upperWeapon)
```

Sets whether the upper weapon is enabled in the saved game data.

Parameters

upperWeapon	Whether the upper weapon is enabled.

6.37.2.23 setUpperWeaponLevel()

```
void setUpperWeaponLevel (
            int upperWeaponLevel)
```

Sets the level of the upper weapon in the saved game data.

Parameters

upperWeaponLevel	The level of the upper weapon.

6.38 SaveData.hpp

Go to the documentation of this file.

```
#pragma once

#include <nlohmann/json.hpp>

using json = nlohmann::json;

void saveData(
    int numberOfPlayers,
    int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score,
    int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score,
    bool upperWeapon, int upperWeaponLevel,
    bool lowerWeapon, int lowerWeaponLevel,
    int typeOfBullet);

void loadSavedData(
    int &numberOfPlayers,
    int &player1Level, int &player1Hp, int &player1HpMax, float &player1Exp, float &player1ExpNext, int &player1Score,
    int &player2Level, int &player2Hp, int &player2HpMax, float &player2Exp, float &player2ExpNext, int &player2Score,
    bool &upperWeapon, int &upperWeaponLevel,
    bool &lowerWeapon, int &lowerWeaponLevel,
    int &typeOfBullet);

void saveDefaultData();

void eraceSavedData();

void setNumberOfPlayers(int numberOfPlayers);
void setPlayer1Level(int player1Level);
void setPlayer1Hp(int player1Hp);
void setPlayer1HpMax(int player1HpMax);
void setPlayer1Exp(float player1Exp);
void setPlayer1ExpNext(float player1ExpNext);
void setPlayer1Score(int player1Score);
void setPlayer2Level(int player2Level);
void setPlayer2Hp(int player2Hp);
void setPlayer2HpMax(int player2HpMax);
void setPlayer2Exp(float player2Exp);
void setPlayer2ExpNext(float player2ExpNext);
void setPlayer2Score(int player2Score);
void setUpperWeapon(bool upperWeapon);
void setUpperWeaponLevel(int upperWeaponLevel);
void setLowerWeapon(bool lowerWeapon);
void setLowerWeaponLevel(int lowerWeaponLevel);
void setTypeOfBullet(int typeOfBullet);
void setContinueGame(bool continueGame);

```

6.39 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Textures.hpp File Reference

This is the header file for the Textures enum, which is used to store the paths to the textures used in the game.

```
#include <unordered_map>
#include <string>
```

Enumerations

- enum **Textures** { **PLAYER** = 0 , **ENEMY_MOVE_LEFT** , **ENEMY_FOLLOW** , **NUM_RESOURCES** }
- enum **SideGuns** { **PEA_SHOOTER** = 0 , **NUM_SIDE_GUNS** }
- enum **Levels** { **SIDE_GUN_LEVEL1** = 0 , **SIDE_GUN_LEVEL2** , **SIDE_GUN_LEVEL3** , **NUM_SIDE_GUN_LEVELS** }

Variables

- const std::unordered_map< Textures, std::string > resourcePaths
- const std::unordered_map< Levels, std::string > PeaShooterUp
- const std::unordered_map< Levels, std::string > PeaShooterDown
- const std::unordered_map< SideGuns, std::unordered_map< Levels, std::string > > SideGunsUp
- const std::unordered_map< SideGuns, std::unordered_map< Levels, std::string > > SideGunsDown

6.39.1 Detailed Description

This is the header file for the Textures enum, which is used to store the paths to the textures used in the game.

Author

Kaloyen

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.39.2 Variable Documentation

6.39.2.1 PeaShooterDown

```
const std::unordered_map<Levels, std::string> PeaShooterDown
```

Initial value:

```
= {
    {SIDE_GUN_LEVEL1, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier1.png"},
    {SIDE_GUN_LEVEL2, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier2.png"},
    {SIDE_GUN_LEVEL3, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier3.png"},
}
```

6.39.2.2 PeaShooterUp

```
const std::unordered_map<Levels, std::string> PeaShooterUp
```

Initial value:

```
= {
    {SIDE_GUN_LEVEL1, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier1.png"},
    {SIDE_GUN_LEVEL2, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier2.png"},
    {SIDE_GUN_LEVEL3, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier3.png"},
}
```

6.39.2.3 resourcePaths

```
const std::unordered_map<Textures, std::string> resourcePaths
```

Initial value:

```
= {

    {PLAYER, "include/Resources/Sprites/Ship.png"},


    {ENEMY_MOVE_LEFT, "include/Resources/Sprites/Enemys/MoveLeft.png"},
    {ENEMY_FOLLOW, "include/Resources/Sprites/Enemys/Follow.png"},
}
```

6.39.2.4 SideGunsDown

```
const std::unordered_map<SideGuns, std::unordered_map<Levels, std::string> > SideGunsDown
```

Initial value:

```
= {

    {PEA_SHOOTER, PeaShooterDown},

}
```

6.39.2.5 SideGunsUp

```
const std::unordered_map<SideGuns, std::unordered_map<Levels, std::string> > SideGunsUp
```

Initial value:

```
= {

    {PEA_SHOOTER, PeaShooterUp},

}
```

6.40 Textures.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include <unordered_map>
00014 #include <string>
00015
00016 enum Textures
00017 {
00018     // Player
00019     PLAYER = 0,
00020
00021     // Enemy
00022     ENEMY_MOVE_LEFT,
00023     ENEMY_FOLLOW,
00024
00025     // Total
00026     NUM_RESOURCES // Renamed from NUM_TEXTURES for clarity
00027 };
00028 enum SideGuns
00029 {
00030     // Player
00031     PEA_SHOOTER = 0,
00032
00033     // Total
00034     NUM_SIDE_GUNS
00035 };
00036
00037 enum Levels
```

```
{
    // Bullets
    SIDE_GUN_LEVEL1 = 0,

    SIDE_GUN_LEVEL2,
    SIDE_GUN_LEVEL3,

    // Total
    NUM_SIDE_GUN_LEVELS
};

const std::unordered_map<Textures, std::string> resourcePaths = {
    // Player
    {PLAYER, "include/Resources/Sprites/Ship.png"},

    // Enemy
    {ENEMY_MOVE_LEFT, "include/Resources/Sprites/Enemys/MoveLeft.png"},
    {ENEMY_FOLLOW, "include/Resources/Sprites/Enemys/Follow.png"},
};

const std::unordered_map<Levels, std::string> PeaShooterUp = {
    {SIDE_GUN_LEVEL1, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier1.png"},
    {SIDE_GUN_LEVEL2, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier2.png"},
    {SIDE_GUN_LEVEL3, "include/Resources/Sprites/SideGuns/Upper/peaShooterTier3.png"},
};

const std::unordered_map<Levels, std::string> PeaShooterDown = {
    {SIDE_GUN_LEVEL1, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier1.png"},
    {SIDE_GUN_LEVEL2, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier2.png"},
    {SIDE_GUN_LEVEL3, "include/Resources/Sprites/SideGuns/Lower/peaShooterTier3.png"},
};

const std::unordered_map<SideGuns, std::unordered_map<Levels, std::string» SideGunsUp = {
    // Player
    {PEA_SHOOTER, PeaShooterUp},

};
const std::unordered_map<SideGuns, std::unordered_map<Levels, std::string» SideGunsDown = {
    // Player
    {PEA_SHOOTER, PeaShooterDown},

};
```

6.41 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/TextTag.hpp File Reference

This is the header file for the TextTag class, which is used to create text tags that can be displayed on the screen.

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <math.h>
#include <SFML/Graphics.hpp>
#include <SFML/System.hpp>
#include <SFML/Window.hpp>
#include <SFML/Audio.hpp>
#include <SFML/Network.hpp>
#include "./Resources/DynamicArray.hpp"
#include "./Resources/SaveData.hpp"
```

Classes

- class TextTag

6.41.1 Detailed Description

This is the header file for the TextTag class, which is used to create text tags that can be displayed on the screen.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.42 TextTag.hpp

Go to the documentation of this file.

```
#pragma once

#include <iostream>
#include <vector>
#include <cstdlib>
#include <math.h>

#include <SFML/Graphics.hpp>
#include <SFML/System.hpp>
#include <SFML/Window.hpp>
#include <SFML/Audio.hpp>
#include <SFML/Network.hpp>

#include "./Resources/DynamicArray.hpp"
#include "./Resources/SaveData.hpp"

using namespace sf;

class TextTag
{
private:
    float dtMultiplier;

    Font *font;
    Text text;

    float speed;
    Vector2f direction;

    float timerMax;
    float timer;

    bool accelerate;

public:
    TextTag(Font *font, std::string text,
            Vector2f position,Vector2f direction, unsigned int size,
            float timerMax, bool accelerate,
            Color color);

    TextTag(const TextTag &other);
    TextTag &operator=(const TextTag &other);
    virtual ~TextTag();

    // Accessors
    inline const float &getTimer() const { return this->timer; }

    void Update(const float &dt);
    void Render(RenderTarget &target);
};
```

6.43 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Weapons/PeaShooter.hpp File Reference

This is the header file for the PeaShooter class, child of Weapon, which is used to create PeaShooter weapons for the player to shoot.

```
#include "Weapon.hpp"
```

Classes

- class PeaShooter

6.43.1 Detailed Description

This is the header file for the PeaShooter class, child of Weapon, which is used to create PeaShooter weapons for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.44 PeaShooter.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "Weapon.hpp"
00014
00015
00016 class PeaShooter : public Weapon
00017 {
00018 private:
00019     int upgradeLevel;
00020     int level;
00021
00022 public:
00023     PeaShooter();
00024     PeaShooter(int level, int UpOrDown);
00025     virtual ~PeaShooter();
00026
00027
00028     // Functions
00029     void Update(Vector2f targetPosition, FloatRect bounds, const float &dt) override;
00030     void Fire(const Vector2f &position, const Vector2f &direction, const Vector2f &scale) override;
00031     void Upgrade() override;
00032
00033 };
```

6.45 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/include/Weapons/Weapon.hpp File Reference

This is the header file for the Weapon class, which is used to create weapons for the player to shoot.

```
#include "../Bullets/Bullet.hpp"
#include "../Resources/Textures.hpp"
```

Classes

- class Weapon

6.45.1 Detailed Description

This is the header file for the Weapon class, which is used to create weapons for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.46 Weapon.hpp

Go to the documentation of this file.

```
00001
00011 #pragma once
00012
00013 #include "../Bullets/Bullet.hpp"
00014 #include "../Resources/Textures.hpp"
00015
00016 class Weapon
00017 {
00018 protected:
00019     float dtMultiplier;
00020
00021     int fireRate;
00022     int fireRateMax;
00023     Sprite sprite;
00024     Texture* weaponTexture;
00025     std::vector<Bullet *> bullets;
00026
00027     int bulletLevel;
00028     int damage;
00029     int damageMax;
```

```
    int UpOrDown;
    int level;

public:
    // Constructor
    Weapon();
    Weapon(int type, int level, int UpOrDown,int fireRateMax = 25, int damageMax = 1);
    virtual ~Weapon();

    // Accessors
    inline std::vector<Bullet*>& getBullets() { return bullets; }
    const Vector2f& getPosition() const { return sprite.getPosition(); }
    const FloatRect getBounds() const { return sprite.getGlobalBounds(); }

    // Setters
    void setPosition(const Vector2f& position) { sprite.setPosition(position); }

    // Virtual functions
    virtual void Update(Vector2f targetPosition, FloatRect bounds, const float &dt);
    virtual void Fire(const Vector2f& position, const Vector2f& direction, const Vector2f& scale);
    virtual void Upgrade();

    // Functions
    void Render(RenderTarget& target);
};
```

6.47 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/Bullet.cpp File Reference

This is the implementation file for the Bullet class, which is used to create bullets for the player to shoot.

```
#include "../../include/Bullets/Bullet.hpp"
#include <iostream>
```

6.47.1 Detailed Description

This is the implementation file for the Bullet class, which is used to create bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.48 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/DarkMatter.cpp File Reference

This is the implementation file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot.

```
#include "../../include/Bullets/DarkMatter.hpp"
```

6.48.1 Detailed Description

This is the implementation file for the DarkMatter class, child of Bullet, which is used to create DarkMatter bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.49 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/LaserBullet.cpp File Reference

This is the implementation file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot.

```
#include "../../include/Bullets/LaserBullet.hpp"
```

6.49.1 Detailed Description

This is the implementation file for the LaserBullet class, child of Bullet, which is used to create Laser bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.50 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/Lightning.cpp File Reference

This is the implementation file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot.

```
#include "../../include/Bullets/Lightning.hpp"
```

6.50.1 Detailed Description

This is the implementation file for the Lightning class, child of Bullet, which is used to create Lightning bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.51 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/NuclierMaterial.cpp File Reference

This is the implementation file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot.

```
#include "../../include/Bullets/NuclierMaterial.hpp"
```

6.51.1 Detailed Description

This is the implementation file for the NuclierMaterial class, child of Bullet, which is used to create NuclierMaterial bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.52 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/PeaBullet.cpp File Reference

This is the implementation file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot.

```
#include "../../include/Bullets/PeaBullet.hpp"
```

6.52.1 Detailed Description

This is the implementation file for the PeaBullet class, child of Bullet, which is used to create Pea bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.53 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/PlanetaryBomb.cpp File Reference

This is the implementation file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot.

```
#include "../../include/Bullets/PlanetaryBomb.hpp"
```

6.53.1 Detailed Description

This is the implementation file for the PlanetaryBomb class, child of Bullet, which is used to create PlanetaryBomb bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.54 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Bullets/Plasma.cpp File Reference

This is the implementation file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot.

```
#include "../../include/Bullets/Plasma.hpp"
```

6.54.1 Detailed Description

This is the implementation file for the Plasma class, child of Bullet, which is used to create Plasma bullets for the player to shoot.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.55 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Enemys/Enemy.cpp File Reference

This is the implementation file for the Enemy class, which is used to create enemies for the player to fight against.

```
#include "../../include/Enemys/Enemy.hpp"
```

6.55.1 Detailed Description

This is the implementation file for the Enemy class, which is used to create enemies for the player to fight against.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.56 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Enemys/Follow.cpp File Reference

This is the implementation file for the FollowEnemy class, a subclass of Enemy, used to create enemies that follow the player.

```
#include "../../include/Enemys/Follow.hpp"
#include <cmath>
```

6.56.1 Detailed Description

This is the implementation file for the FollowEnemy class, a subclass of Enemy, used to create enemies that follow the player.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.57 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Enemys/MoveLeft.cpp File Reference

This is the implementation file for the MoveLeftEnemy class, a subclass of Enemy, used to create enemies that move left.

```
#include "../../include/Enemys/MoveLeft.hpp"
```

6.57.1 Detailed Description

This is the implementation file for the MoveLeftEnemy class, a subclass of Enemy, used to create enemies that move left.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.58 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Game.cpp File Reference

This is the implementation file for the Game class, which is used to manage the game logic.

```
#include "../include/Game.hpp"
#include <stdexcept>
#include <cassert>
```

6.58.1 Detailed Description

This is the implementation file for the Game class, which is used to manage the game logic.

Author

Kaloyan

The Game class is responsible for managing the game logic, including player and enemy interactions, bullet collisions, and UI elements.

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.59 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/main.cpp File Reference

This is the main file for the game.

```
#include <nlohmann/json.hpp>
#include <fstream>
#include <iostream>
#include "../include/Game.hpp"
#include "../include/Menu.hpp"
#include "../include/Resources/SaveData.hpp"
```

Functions

- int main ()

6.59.1 Detailed Description

This is the main file for the game.

Author

Kaloyan

This file contains the main function which initializes the game window, loads saved data, creates game and menu objects, and starts the main game loop.

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.59.2 Function Documentation

6.59.2.1 main()

```
int main ()
```

Seed the random number generator

Create a fullscreen window

Handle window creation failure

Load saved data

Player 1 data

Player 2 data

Weapons data

Create the game object

Create the menu object

Main game loop

Extract current game state before saving

Save current game state

Update dt

Update Handle menu input

Handle game input

Update the game

Render

6.60 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Menu.cpp File Reference

This is the implementation file for the Menu class, which is used to create and manage the game menu.

```
#include "../include/Menu.hpp"
#include <iostream>
#include <fstream>
```

6.60.1 Detailed Description

This is the implementation file for the Menu class, which is used to create and manage the game menu.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.61 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Player.cpp File Reference

Implementation file for the Player class.

```
#include "../include/Player.hpp"
```

6.61.1 Detailed Description

Implementation file for the Player class.

Author

Kaloyan

Version

0.1

Date

2024-05-30

This file defines the member functions of the Player class, which is used to create player objects.

Copyright

Copyright (c) 2024

6.62 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Resources/Button.cpp File Reference

This is the implementation file for the Button class, which is used to create buttons for the game.

```
#include "../../include/Resources/Button.hpp"
#include <iostream>
```

6.62.1 Detailed Description

This is the implementation file for the Button class, which is used to create buttons for the game.

Author

Kaloyan

Version

0.1

Date

2024-05-30

This file provides the implementation of the Button class, which allows creating interactive buttons in the game.

Copyright

Copyright (c) 2024

6.63 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Resources/SaveData.cpp File Reference

This is the implementation file for the SaveData functions, which are used to save and load game data.

```
#include "../../include/Resources/SaveData.hpp"
#include <fstream>
#include <iostream>
```

Functions

- void saveData (int numberOfPlayers, int player1Level, int player1Hp, int player1HpMax, float player1Exp, float player1ExpNext, int player1Score, int player2Level, int player2Hp, int player2HpMax, float player2Exp, float player2ExpNext, int player2Score, bool upperWeapon, int upperWeaponLevel, bool lowerWeapon, int lower↩ WeaponLevel, int typeOfBullet)

  Saves game data to a JSON file.
- void loadSavedData (int &numberOfPlayers, int &player1Level, int &player1Hp, int &player1HpMax, float &player1Exp, float &player1ExpNext, int &player1Score, int &player2Level, int &player2Hp, int &player2↩ HpMax, float &player2Exp, float &player2ExpNext, int &player2Score, bool &upperWeapon, int &upper↩ WeaponLevel, bool &lowerWeapon, int &lowerWeaponLevel, int &typeOfBullet)

  Loads game data from a JSON file.
- void saveDefaultData ()

  Saves default game data to a JSON file.
- void eraceSavedData ()

  Erases saved game data by replacing it with default data.
- json readJsonFromFile (const std::string &fileName)

  Helper function to read JSON data from a file.
- void writeJsonToFile (const std::string &fileName, const json &savedData)

  Helper function to write JSON data to a file.
- void setNumberOfPlayers (int numberOfPlayers)

  Sets the number of players in the saved game data.
- void setPlayer1Level (int player1Level)

  Sets the level of player 1 in the saved game data.
- void setPlayer1Hp (int player1Hp)

  Sets the HP of player 1 in the saved game data.
- void setPlayer1HpMax (int player1HpMax)

  Sets the maximum HP of player 1 in the saved game data.
- void setPlayer1Exp (float player1Exp)

  Sets the experience points of player 1 in the saved game data.
- void setPlayer1ExpNext (float player1ExpNext)

  Sets the next level experience points of player 1 in the saved game data.
- void setPlayer1Score (int player1Score)

  Sets the score of player 1 in the saved game data.
- void setPlayer2Level (int player2Level)

  Sets the level of player 2 in the saved game data.
- void setPlayer2Hp (int player2Hp)

  Sets the HP of player 2 in the saved game data.
- void setPlayer2HpMax (int player2HpMax)

  Sets the maximum HP of player 2 in the saved game data.
- void setPlayer2Exp (float player2Exp)

  Sets the experience points of player 2 in the saved game data.
- void setPlayer2ExpNext (float player2ExpNext)

  Sets the next level experience points of player 2 in the saved game data.
- void setPlayer2Score (int player2Score)

  Sets the score of player 2 in the saved game data.
- void setUpperWeapon (bool upperWeapon)

  Sets whether the upper weapon is enabled in the saved game data.
- void setUpperWeaponLevel (int upperWeaponLevel)

  Sets the level of the upper weapon in the saved game data.
- void setLowerWeapon (bool lowerWeapon)

  Sets whether the lower weapon is enabled in the saved game data.

- void setLowerWeaponLevel (int lowerWeaponLevel)

  Sets the level of the lower weapon in the saved game data.
- void setTypeOfBullet (int typeOfBullet)

  Sets the type of bullet in the saved game data.
- void setContinueGame (bool continueGame)

  Sets whether the game continues from the saved state in the saved game data.

6.63.1 Detailed Description

This is the implementation file for the SaveData functions, which are used to save and load game data.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.63.2 Function Documentation

6.63.2.1 eraceSavedData()

```
void eraceSavedData ()
```

Erases saved game data by replacing it with default data.

Create default data

Open file to save default data

Write default data to file

6.63.2.2 loadSavedData()

```
void loadSavedData (
            int & numberOfPlayers,
            int & player1Level,
            int & player1Hp,
            int & player1HpMax,
            float & player1Exp,
            float & player1ExpNext,
            int & player1Score,
            int & player2Level,
            int & player2Hp,
            int & player2HpMax,
            float & player2Exp,
            float & player2ExpNext,
            int & player2Score,
            bool & upperWeapon,
            int & upperWeaponLevel,
            bool & lowerWeapon,
            int & lowerWeaponLevel,
            int & typeOfBullet)
```

Loads game data from a JSON file.

Parameters

numberOfPlayers	The number of players.
player1Level	Player 1's level.
player1Hp	Player 1's health points.
player1HpMax	Player 1's maximum health points.
player1Exp	Player 1's experience.
player1ExpNext	Player 1's experience required for next level.
player1Score	Player 1's score.
player2Level	Player 2's level.
player2Hp	Player 2's health points.
player2HpMax	Player 2's maximum health points.
player2Exp	Player 2's experience.
player2ExpNext	Player 2's experience required for next level.
player2Score	Player 2's score.
upperWeapon	Whether the upper weapon is enabled.
upperWeaponLevel	The level of the upper weapon.
lowerWeapon	Whether the lower weapon is enabled.
lowerWeaponLevel	The level of the lower weapon.
typeOfBullet	The type of bullet.

Open file to load data

Read data from file

Extract game data

6.63.2.3 readJsonFromFile()

```
json readJsonFromFile (
            const std::string & fileName)
```

Helper function to read JSON data from a file.

Parameters

fileName	The name of the file.

Returns

json The JSON data.

6.63.2.4 saveData()

```
void saveData (
            int numberOfPlayers,
            int player1Level,
            int player1Hp,
            int player1HpMax,
            float player1Exp,
            float player1ExpNext,
            int player1Score,
            int player2Level,
            int player2Hp,
            int player2HpMax,
            float player2Exp,
            float player2ExpNext,
            int player2Score,
            bool upperWeapon,
            int upperWeaponLevel,
            bool lowerWeapon,
            int lowerWeaponLevel,
            int typeOfBullet)
```

Saves game data to a JSON file.

Parameters

numberOfPlayers	The number of players.
player1Level	Player 1's level.
player1Hp	Player 1's health points.
player1HpMax	Player 1's maximum health points.
player1Exp	Player 1's experience.
player1ExpNext	Player 1's experience required for next level.
player1Score	Player 1's score.
player2Level	Player 2's level.
player2Hp	Player 2's health points.
player2HpMax	Player 2's maximum health points.
player2Exp	Player 2's experience.
player2ExpNext	Player 2's experience required for next level.
player2Score	Player 2's score.
upperWeapon	Whether the upper weapon is enabled.
upperWeaponLevel	The level of the upper weapon.
lowerWeapon	Whether the lower weapon is enabled.
lowerWeaponLevel	The level of the lower weapon.
typeOfBullet	The type of bullet.

6.63.2.5 saveDefaultData()

```
void saveDefaultData ()
```

Saves default game data to a JSON file.

Open file to load data

Read data from file

Update default data

Save default data

6.63.2.6 setContinueGame()

```
void setContinueGame (
            bool continueGame)
```

Sets whether the game continues from the saved state in the saved game data.

Parameters

continueGame	Whether the game continues.

6.63.2.7 setLowerWeapon()

```
void setLowerWeapon (
            bool lowerWeapon)
```

Sets whether the lower weapon is enabled in the saved game data.

Parameters

lowerWeapon	Whether the lower weapon is enabled.

6.63.2.8 setLowerWeaponLevel()

```
void setLowerWeaponLevel (
            int lowerWeaponLevel)
```

Sets the level of the lower weapon in the saved game data.

Parameters

lowerWeaponLevel	The level of the lower weapon.

6.63.2.9 setNumberOfPlayers()

```
void setNumberOfPlayers (
            int numberOfPlayers)
```

Sets the number of players in the saved game data.

Parameters

numberOfPlayers	The number of players.

6.63.2.10 setPlayer1Exp()

```
void setPlayer1Exp (
            float player1Exp)
```

Sets the experience points of player 1 in the saved game data.

Parameters

player1Exp	The experience points of player 1.

6.63.2.11 setPlayer1ExpNext()

```
void setPlayer1ExpNext (
            float player1ExpNext)
```

Sets the next level experience points of player 1 in the saved game data.

Parameters

player1ExpNext	The next level experience points of player 1.

6.63.2.12 setPlayer1Hp()

```
void setPlayer1Hp (
            int player1Hp)
```

Sets the HP of player 1 in the saved game data.

Parameters

player1Hp	The HP of player 1.

6.63.2.13 setPlayer1HpMax()

```
void setPlayer1HpMax (
            int player1HpMax)
```

Sets the maximum HP of player 1 in the saved game data.

Parameters

player1HpMax	The maximum HP of player 1.

6.63.2.14 setPlayer1Level()

```
void setPlayer1Level (
            int player1Level)
```

Sets the level of player 1 in the saved game data.

Parameters

player1Level	The level of player 1.

6.63.2.15 setPlayer1Score()

```
void setPlayer1Score (
            int player1Score)
```

Sets the score of player 1 in the saved game data.

Parameters

player1Score	The score of player 1.

6.63.2.16 setPlayer2Exp()

```
void setPlayer2Exp (
            float player2Exp)
```

Sets the experience points of player 2 in the saved game data.

Parameters

player2Exp	The experience points of player 2.

6.63.2.17 setPlayer2ExpNext()

```
void setPlayer2ExpNext (
            float player2ExpNext)
```

Sets the next level experience points of player 2 in the saved game data.

Parameters

player2ExpNext	The next level experience points of player 2.

6.63.2.18 setPlayer2Hp()

```
void setPlayer2Hp (
            int player2Hp)
```

Sets the HP of player 2 in the saved game data.

Parameters

player2Hp	The HP of player 2.

6.63.2.19 setPlayer2HpMax()

```
void setPlayer2HpMax (
            int player2HpMax)
```

Sets the maximum HP of player 2 in the saved game data.

Parameters

player2HpMax	The maximum HP of player 2.

6.63.2.20 setPlayer2Level()

```
void setPlayer2Level (
            int player2Level)
```

Sets the level of player 2 in the saved game data.

Parameters

player2Level	The level of player 2.

6.63.2.21 setPlayer2Score()

```
void setPlayer2Score (
            int player2Score)
```

Sets the score of player 2 in the saved game data.

Parameters

player2Score	The score of player 2.

6.63.2.22 setTypeOfBullet()

```
void setTypeOfBullet (
            int typeOfBullet)
```

Sets the type of bullet in the saved game data.

Parameters

typeOfBullet	The type of bullet.

6.63.2.23 setUpperWeapon()

```
void setUpperWeapon (
            bool upperWeapon)
```

Sets whether the upper weapon is enabled in the saved game data.

Parameters

upperWeapon	Whether the upper weapon is enabled.

6.63.2.24 setUpperWeaponLevel()

```
void setUpperWeaponLevel (
            int upperWeaponLevel)
```

Sets the level of the upper weapon in the saved game data.

Parameters

upperWeaponLevel	The level of the upper weapon.

6.63.2.25 writeJsonToFile()

```
void writeJsonToFile (
            const std::string & fileName,
            const json & savedData)
```

Helper function to write JSON data to a file.

Parameters

fileName	The name of the file.
savedData	The JSON data to write.

6.64 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/TextTag.cpp File Reference

Implementation file for the TextTag class.

```
#include "../include/TextTag.hpp"
```

6.64.1 Detailed Description

Implementation file for the TextTag class.

Author

Kaloyan

Version

0.1

Date

2024-05-30

This file defines the member functions of the TextTag class, which is used to create text tags that can be displayed on the screen.

Copyright

Copyright (c) 2024

6.65 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Weapons/PeaShooter.cpp File Reference

This is the implementation file for the PeaShooter class, a child of Weapon, used to create the pea shooter weapon for the player.

```
#include "../../include/Weapons/PeaShooter.hpp"
#include "../../include/Bullets/PeaBullet.hpp"
#include <iostream>
```

6.65.1 Detailed Description

This is the implementation file for the PeaShooter class, a child of Weapon, used to create the pea shooter weapon for the player.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

6.66 /Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/↩ SpaceShooter/src/Weapons/Weapon.cpp File Reference

This is the implementation file for the Weapon class, which is used to create weapons for the player.

```
#include "../../include/Weapons/Weapon.hpp"
#include <iostream>
```

6.66.1 Detailed Description

This is the implementation file for the Weapon class, which is used to create weapons for the player.

Author

Kaloyan

Version

0.1

Date

2024-05-30

Copyright

Copyright (c) 2024

Index

/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Bullet.hpp, 65, 66
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/DarkMatter.hpp, 66, 67
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/LaserBullet.hpp, 67, 68
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Lightning.hpp, 68, 69
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/NuclierMaterial.hpp, 69, 70
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PeaBullet.hpp, 70, 71
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/PlanetaryBomb.hpp, 71, 72
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Bullets/Plasma.hpp, 72, 73
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Enemy.hpp, 73, 74
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/Follow.hpp, 75, 76
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Enemys/MoveLeft.hpp, 76, 77
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Game.hpp, 77, 78
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Menu.hpp, 80, 81
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Player.hpp, 81, 83
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/BulletResources.hpp, 85, 90
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Button.hpp, 92, 93
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/DynamicArray.hpp, 93, 94
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Fonts.hpp, 96, 97
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/SaveData.hpp, 97, 107
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Resources/Textures.hpp, 107, 109
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/TextTag.hpp, 110, 111
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/PeaShooter.hpp, 112
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/include/Weapons/Weapon.hpp, 113
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/Bullet.cpp, 114
/Users/kaloyanbakardzhiev/Documents/Uni/OOP/Project/sfml/SpaceShooter/src/Bullets/DarkMatter.cpp,

~Game
Game, 39
~PeaShooter
PeaShooter, 49

addButton
Menu, 40
addButtonToBeginMenu
Menu, 40
addWeapon
Player, 57
Bullet, 11

Bullet, 12
Move, 12
Render, 12
setDamage, 12
setTexture, 13
Update, 13
BulletResources.hpp
resourcePathsBulletsDamage, 86
resourcePathsDamageMaps, 86
resourcePathsDarkMatterBullet, 86
resourcePathsDarkMatterBulletDamage, 87
resourcePathsLaserBullets, 87
resourcePathsLightningBullets, 87
resourcePathsLightningBulletsDamage, 87
resourcePathsMaps, 87
resourcePathsNuclierMaterialBullet, 88
resourcePathsNuclierMaterialBulletDamage, 88
resourcePathsPeaShooterBullets, 88
resourcePathsPeaShooterBulletsDamage, 88
resourcePathsPlanetaryBombBullets, 88
resourcePathsPlanetaryBombBulletsDamage, 89
resourcePathsPlasmaBullets, 89
resourcePathsPlasmaBulletsDamage, 89
Button, 13
Button, 14
changeColor, 15
getText, 15
isClicked, 16
render, 16
setOnClick, 16
update, 17

changeColor
Button, 15
CombatUpdate
Player, 57
createPlayer
Game, 29

DarkMatter, 17
DarkMatter, 18
Update, 19
DynamicArray< T >, 19

Enemy, 20
Enemy, 21
Move, 21
Render, 21
TakeDamage, 21
Update, 23
eraceSavedData
SaveData.cpp, 126
SaveData.hpp, 99
exit
Game, 30

Fire
PeaShooter, 50
Weapon, 63
FollowEnemy, 23
FollowEnemy, 25
Move, 25
Fonts.hpp
resourcePathsFonts, 97

Game, 26
~Game, 29
createPlayer, 29
exit, 30
Game, 28
getGameState, 30
initUI, 31
render, 32
renderUI, 32
restart, 33
update, 33
updateUI, 34
getBullet
Player, 57
getGameState
Game, 30
getText
Button, 15

initUI
Game, 31
isButtonPressed
Menu, 40
isClicked
Button, 16

LaserBullet, 35
LaserBullet, 36
Update, 36
Lightning, 37
Lightning, 38
Update, 38
loadSavedData
SaveData.cpp, 126
SaveData.hpp, 99

main
main.cpp, 122
main.cpp
main, 122
Menu, 39
addButton, 40
addButtonToBeginMenu, 40
isButtonPressed, 40
Menu, 39
render, 41
resetAllColor, 41
resetColor, 41
resetLowerLevelColor, 41
resetUpperLevelColor, 42
update, 42
Move
Bullet, 12

Enemy, 21
FollowEnemy, 25
MoveLeftEnemy, 44
Player, 57
MoveLeftEnemy, 42
Move, 44
MoveLeftEnemy, 44

NuclierMaterial, 45
NuclierMaterial, 46
Update, 46

operator=
TextTag, 61

PeaBullet, 46
PeaBullet, 47
Update, 48
PeaShooter, 48
~PeaShooter, 49
Fire, 50
PeaShooter, 49
Update, 50
Upgrade, 50
PeaShooterDown
Textures.hpp, 108
PeaShooterUp
Textures.hpp, 108
PlanetaryBomb, 51
PlanetaryBomb, 52
Update, 52
Plasma, 52
Plasma, 53
Update, 54
Player, 54
addWeapon, 57
CombatUpdate, 57
getBullet, 57
Move, 57
Player, 56
removeBullet, 58
Render, 58
setBulletType, 58
TakeDamage, 59
Update, 59
UpdateAccessories, 59
updateLevelingSystem, 60

readJsonFromFile
SaveData.cpp, 127
README, 1
removeBullet
Player, 58
Render
Bullet, 12
Enemy, 21
Player, 58
TextTag, 61
Weapon, 63
render
Button, 16
Game, 32
Menu, 41
renderUI
Game, 32
resetAllColor
Menu, 41
resetColor
Menu, 41
resetLowerLevelColor
Menu, 41
resetUpperLevelColor
Menu, 42
resourcePaths
Textures.hpp, 108
resourcePathsBulletsDamage
BulletResources.hpp, 86
resourcePathsDamageMaps
BulletResources.hpp, 86
resourcePathsDarkMatterBullet
BulletResources.hpp, 86
resourcePathsDarkMatterBulletDamage
BulletResources.hpp, 87
resourcePathsFonts
Fonts.hpp, 97
resourcePathsLaserBullets
BulletResources.hpp, 87
resourcePathsLightningBullets
BulletResources.hpp, 87
resourcePathsLightningBulletsDamage
BulletResources.hpp, 87
resourcePathsMaps
BulletResources.hpp, 87
resourcePathsNuclierMaterialBullet
BulletResources.hpp, 88
resourcePathsNuclierMaterialBulletDamage
BulletResources.hpp, 88
resourcePathsPeaShooterBullets
BulletResources.hpp, 88
resourcePathsPeaShooterBulletsDamage
BulletResources.hpp, 88
resourcePathsPlanetaryBombBullets
BulletResources.hpp, 88
resourcePathsPlanetaryBombBulletsDamage
BulletResources.hpp, 89
resourcePathsPlasmaBullets
BulletResources.hpp, 89
resourcePathsPlasmaBulletsDamage
BulletResources.hpp, 89
restart
Game, 33

saveData
SaveData.cpp, 129
SaveData.hpp, 100
SaveData.cpp
eraceSavedData, 126
loadSavedData, 126

readJsonFromFile, 127
saveData, 129
saveDefaultData, 129
setContinueGame, 130
setLowerWeapon, 130
setLowerWeaponLevel, 130
setNumberOfPlayers, 130
setPlayer1Exp, 131
setPlayer1ExpNext, 131
setPlayer1Hp, 131
setPlayer1HpMax, 131
setPlayer1Level, 132
setPlayer1Score, 132
setPlayer2Exp, 132
setPlayer2ExpNext, 132
setPlayer2Hp, 133
setPlayer2HpMax, 133
setPlayer2Level, 133
setPlayer2Score, 133
setTypeOfBullet, 134
setUpperWeapon, 134
setUpperWeaponLevel, 134
writeJsonToFile, 134
SaveData.hpp
eraceSavedData, 99
loadSavedData, 99
saveData, 100
saveDefaultData, 101
setContinueGame, 102
setLowerWeapon, 102
setLowerWeaponLevel, 102
setNumberOfPlayers, 102
setPlayer1Exp, 103
setPlayer1ExpNext, 103
setPlayer1Hp, 103
setPlayer1HpMax, 103
setPlayer1Level, 104
setPlayer1Score, 104
setPlayer2Exp, 104
setPlayer2ExpNext, 104
setPlayer2Hp, 105
setPlayer2HpMax, 105
setPlayer2Level, 105
setPlayer2Score, 105
setTypeOfBullet, 106
setUpperWeapon, 106
setUpperWeaponLevel, 106
saveDefaultData
SaveData.cpp, 129
SaveData.hpp, 101
setBulletType
Player, 58
setContinueGame
SaveData.cpp, 130
SaveData.hpp, 102
setDamage
Bullet, 12
setLowerWeapon
SaveData.cpp, 130
SaveData.hpp, 102
setLowerWeaponLevel
SaveData.cpp, 130
SaveData.hpp, 102
setNumberOfPlayers
SaveData.cpp, 130
SaveData.hpp, 102
setOnClick
Button, 16
setPlayer1Exp
SaveData.cpp, 131
SaveData.hpp, 103
setPlayer1ExpNext
SaveData.cpp, 131
SaveData.hpp, 103
setPlayer1Hp
SaveData.cpp, 131
SaveData.hpp, 103
setPlayer1HpMax
SaveData.cpp, 131
SaveData.hpp, 103
setPlayer1Level
SaveData.cpp, 132
SaveData.hpp, 104
setPlayer1Score
SaveData.cpp, 132
SaveData.hpp, 104
setPlayer2Exp
SaveData.cpp, 132
SaveData.hpp, 104
setPlayer2ExpNext
SaveData.cpp, 132
SaveData.hpp, 104
setPlayer2Hp
SaveData.cpp, 133
SaveData.hpp, 105
setPlayer2HpMax
SaveData.cpp, 133
SaveData.hpp, 105
setPlayer2Level
SaveData.cpp, 133
SaveData.hpp, 105
setPlayer2Score
SaveData.cpp, 133
SaveData.hpp, 105
setTexture
Bullet, 13
setTypeOfBullet
SaveData.cpp, 134
SaveData.hpp, 106
setUpperWeapon
SaveData.cpp, 134
SaveData.hpp, 106
setUpperWeaponLevel
SaveData.cpp, 134
SaveData.hpp, 106
SideGunsDown

Textures.hpp, 109
SideGunsUp
Textures.hpp, 109

TakeDamage
Enemy, 21
Player, 59
TextTag, 60
operator=, 61
Render, 61
TextTag, 60, 61
Update, 62
Textures.hpp
PeaShooterDown, 108
PeaShooterUp, 108
resourcePaths, 108
SideGunsDown, 109
SideGunsUp, 109

Update
Bullet, 13
DarkMatter, 19
Enemy, 23
LaserBullet, 36
Lightning, 38
NuclierMaterial, 46
PeaBullet, 48
PeaShooter, 50
PlanetaryBomb, 52
Plasma, 54
Player, 59
TextTag, 62
Weapon, 64
update
Button, 17
Game, 33
Menu, 42
UpdateAccessories
Player, 59
updateLevelingSystem
Player, 60
updateUI
Game, 34
Upgrade
PeaShooter, 50
Weapon, 64

Weapon, 62
Fire, 63
Render, 63
Update, 64
Upgrade, 64
Weapon, 63
writeJsonToFile
SaveData.cpp, 134